
User Manual

for MPC560XB CAN Driver

Document Number: UM13CANASR4.0R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	15
2.2	Overview.....	16
2.3	About this Manual.....	16
2.4	Acronyms and Definitions.....	17
2.5	Reference List.....	17
2.6	Symbolic Names DISCLAIMER.....	17
Chapter 3		
Driver		
3.1	Requirements.....	19
3.2	Driver Design Summary.....	19
3.3	Calling Driver APIs.....	20
3.4	Deviation from Requirements.....	27
3.5	Function Definitions.....	29
3.5.1	Function Can_Init.....	30
3.5.2	Function Can_GetVersionInfo.....	30
3.5.3	Function Can_SetControllerMode.....	31
3.5.4	Function Can_DisableControllerInterrupts.....	32
3.5.5	Function Can_EnableControllerInterrupts.....	32
3.5.6	Function Can_Write.....	33
3.5.7	Function Can_CheckWakeup.....	34
3.5.8	Function Can_MainFunction_Write.....	35
3.5.9	Function Can_MainFunction_Read.....	35
3.5.10	Function Can_MainFunction_BusOff.....	36
3.5.11	Function Can_MainFunction_Wakeup.....	36

Section number	Title	Page
3.5.12	Function Can_MainFunction_Mode.....	37
3.5.13	Function Can_AbortMb.....	37
3.5.14	Function Can_SetClockMode.....	38
3.5.15	Function Can_ChangeBaudrate.....	38
3.5.16	Function Can_CheckBaudrate.....	39
3.5.17	Function Can_GetErrorStatus.....	40
3.6	Structure Definitions.....	40
3.6.1	Structure Can_ConfigType.....	40
3.6.2	Structure Can_ControllerDescriptorType.....	41
3.6.3	Structure Can_ControllerBaudrateConfigType.....	42
3.6.4	Structure Can_ControllerStatusType.....	43
3.6.5	Structure Can_MBCConfigContainerType.....	44
3.6.6	Structure Can_MBCConfigObjectType.....	44
3.6.7	Structure Can_PduType.....	45
3.6.8	Structure Can_RxFifoTableIdConfigType.....	46
3.7	Define Definitions.....	46
3.7.1	Define CAN_API_ENABLE_ABORT_MB.....	46
3.7.2	Define CAN_BCC_SUPPORT_ENABLE.....	46
3.7.3	Define CAN_BUSOFFPOLL_SUPPORTED.....	47
3.7.4	Define CAN_DEV_ERROR_DETECT.....	47
3.7.5	Define CAN_DUAL_CLOCK_MODE.....	47
3.7.6	Define CAN_E_DEFAULT.....	47
3.7.7	Define CAN_E_DATALOST.....	47
3.7.8	Define CAN_E_PARAM_CONTROLLER.....	48
3.7.9	Define CAN_E_PARAM_DLC.....	48
3.7.10	Define CAN_E_PARAM_HANDLE.....	48
3.7.11	Define CAN_E_PARAM_POINTER.....	48
3.7.12	Define CAN_E_TRANSITION.....	48
3.7.13	Define CAN_E_UNINIT.....	48

Section number	Title	Page
3.7.14	Define CAN_E_PARAM_BAUDRATE.....	48
3.7.15	Define CAN_ERROR_NOTIFICATION_ENABLE.....	49
3.7.16	Define CAN_EXTENDEDID.....	49
3.7.17	Define CAN_HW_TRANSMIT_CANCELLATION.....	49
3.7.18	Define CAN_IDENTICAL_ID_CANCELLATION.....	49
3.7.19	Define CAN_INIT_CONFIG_CONTROLLERS_PC.....	49
3.7.20	Define CAN_INIT_CONFIG_PC_DEFINES.....	49
3.7.21	Define CAN_ISROPTCODESIZE.....	50
3.7.22	Define CAN_MAINFUNCTION_MODE_PERIOD.....	50
3.7.23	Define CAN_MAINFUNCTION_PERIOD_BUSOFF.....	50
3.7.24	Define CAN_MAINFUNCTION_PERIOD_READ.....	50
3.7.25	Define CAN_MAINFUNCTION_PERIOD_WAKEUP.....	50
3.7.26	Define CAN_MAINFUNCTION_PERIOD_WRITE.....	50
3.7.27	Define CAN_MAXCTRL_SUPPORTED.....	51
3.7.28	Define CAN_MAXMB_SUPPORTED.....	51
3.7.29	Define CAN_MBCOUNTEXTENSION.....	51
3.7.30	Define CAN_MIX_MB_SUPPORT.....	51
3.7.31	Define CAN_MULTIPLEXED_TRANSMISSION.....	51
3.7.32	Define CAN_PRECOMPILE_SUPPORT.....	51
3.7.33	Define CAN_RXFIFO_ENABLE.....	51
3.7.34	Define CAN_RXFIFO_EVENT_UNIFIED.....	52
3.7.35	Define CAN_RXPOLL_SUPPORTED.....	52
3.7.36	Define CAN_SID_ABORT_MB.....	52
3.7.37	Define CAN_SID_CBK_CHECK_WAKEUP.....	52
3.7.38	Define CAN_SID_DISABLE_CONTROLLER_INTERRUPTS.....	52
3.7.39	Define CAN_SID_ENABLE_CONTROLLER_INTERRUPTS.....	52
3.7.40	Define CAN_SID_GET_VERSION_INFO.....	53
3.7.41	Define CAN_SID_INIT.....	53
3.7.42	Define CAN_SID_INIT_CONTROLLER.....	53

Section number	Title	Page
3.7.43	Define CAN_SID_MAIN_FUNCTION_BUS_OFF.....	53
3.7.44	Define CAN_SID_MAIN_FUNCTION_MODE.....	53
3.7.45	Define CAN_SID_MAIN_FUNCTION_READ.....	53
3.7.46	Define CAN_SID_MAIN_FUNCTION_WAKEUP.....	54
3.7.47	Define CAN_SID_MAIN_FUNCTION_WRITE.....	54
3.7.48	Define CAN_SID_SET_CONTROLLER_MODE.....	54
3.7.49	Define CAN_SID_SETCLOCKMODE.....	54
3.7.50	Define CAN_SID_WRITE.....	54
3.7.51	Define CAN_SID_CHANGE_BAUDRATE.....	54
3.7.52	Define CAN_SID_CHECK_BAUDRATE.....	55
3.7.53	Define CAN_TIMEOUT_DURATION.....	55
3.7.54	Define CAN_TXPOLL_SUPPORTED.....	55
3.7.55	Define CAN_VERSION_INFO_API.....	55
3.7.56	Define CAN_WAKEUP_SUPPORT.....	55
3.7.57	Define CAN_WAKEUPPOLL_SUPPORTED.....	55
3.7.58	Define Can_Abort_Only_One_MB.....	56
3.7.59	Define CAN_MAXCTRL_CONFIGURED.....	56
3.7.60	Define CAN_MAXMB_CONFIGURED.....	56
3.8	Enum Definitions.....	56
3.8.1	Enumeration Can_ClockMode.....	56
3.8.2	Enumeration Can_ControllerStateType.....	57
3.8.3	Enumeration Can_ObjType.....	57
3.8.4	Enumeration Can_ReturnType.....	57
3.8.5	Enumeration Can_StateTransitionType.....	58
3.8.6	Enumeration Can_StatusType.....	58
3.8.7	Enumeration CanIdType.....	58
3.9	Typedef Definitions.....	59
3.9.1	Typedef Can_HwHandleType.....	59
3.9.2	Typedef Can_IdPtrType.....	59

Section number	Title	Page
3.9.3	Typedef Can_IdType.....	59
3.9.4	Typedef Can_PCallBackType.....	60
3.9.5	Typedef Can_PtrControllerDescriptorType.....	60
3.9.6	Typedef Can_PtrMBConfigContainerType.....	60
3.10	Configuration Parameters.....	60
3.10.1	Can-General Parameters.....	61
3.10.1.1	IMPLEMENTATION_CONFIG_VARIANT.....	61
3.10.1.2	CanDevErrorDetection.....	62
3.10.1.3	CanVersionInfoApi.....	62
3.10.1.4	CanIndex.....	63
3.10.1.5	CanMainFunctionBusOffPeriod.....	63
3.10.1.6	CanMainFunctionModePeriod.....	63
3.10.1.7	CanMainFunctionWakeupPeriod.....	64
3.10.1.8	CanIdenticalIdCancellation.....	64
3.10.1.9	CanMultiplexedTransmission.....	65
3.10.1.10	CanHardwareCancellation.....	65
3.10.1.11	CanTimeoutDurationFactor.....	66
3.10.1.12	CanCrystalFrequencyHz.....	66
3.10.1.13	CanLPduReceiveCalloutFunction.....	67
3.10.1.14	CanCodeSizeOptimization.....	67
3.10.1.15	CanExtendedIdSupport.....	67
3.10.1.16	CanMBCountExtensionSupport.....	68
3.10.1.17	CanApiEnableMbAbort.....	68
3.10.1.18	CanEnableDualClockMode.....	69
3.10.1.19	CanCounterRef.....	69
3.10.1.20	CanMainFunctionReadPeriodRef.....	70
3.10.1.21	CanMainFunctionWritePeriodRef.....	70
3.10.1.22	CanSupportTTCANRef.....	70
3.10.1.23	CanMainFunctionRWPeriods.....	71

Section number	Title	Page
3.10.1.24	Can_ChangeBaudrate.....	72
3.10.1.25	CanAbortOnlyOneMB.....	73
3.10.2	Can-Controller Parameters.....	73
3.10.2.1	CanHwChannel.....	74
3.10.2.2	CanControllerActivation.....	75
3.10.2.3	CanControllerBaseAddress.....	76
3.10.2.4	CanControllerId.....	76
3.10.2.5	CanWakeupSupport.....	76
3.10.2.6	CanRxProcessing.....	77
3.10.2.7	CanTxProcessing.....	77
3.10.2.8	CanBusOffProcessing.....	78
3.10.2.9	CanWakeupProcessing.....	78
3.10.2.10	CanListenOnlyMode.....	79
3.10.2.11	CanLoopBackMode.....	80
3.10.2.12	CanSoftwareBusOffRecovery.....	80
3.10.2.13	CanAutoBusOffRecovery.....	81
3.10.2.14	CanTripleSamplingEnable.....	81
3.10.2.15	CanWakeUpSourceFilter.....	82
3.10.2.16	CanLowestBuffTransmitFirst.....	83
3.10.2.17	CanLocalPriorityEn.....	83
3.10.2.18	CanWarningEnable.....	84
3.10.2.19	CanClockFromBus.....	84
3.10.2.20	CanCpuClockRef.....	85
3.10.2.21	CanControllerRXFifoEnable.....	85
3.10.2.22	CanRxFifoWarningNotification.....	86
3.10.2.23	CanRxFifoOverflowNotification.....	86
3.10.2.24	CanErrorControllerNotification.....	87
3.10.2.25	CanWakeupSourceRef.....	87
3.10.2.26	CanCpuClockRef_Alternate.....	87

Section number	Title	Page
3.10.2.27	CanBccSupport.....	88
3.10.2.28	CanErrorControllerNotifEn.....	88
3.10.2.29	CanControllerDefaultBaudrate.....	89
3.10.3	CanFilterMask.....	89
3.10.3.1	CanFilterMaskValue.....	90
3.10.4	Can RxFifo.....	90
3.10.4.1	CanControllerIDAcceptanceMode.....	93
3.10.4.2	CanIDValue0.....	93
3.10.4.3	CanIDValue1.....	94
3.10.4.4	CanIDValue2.....	95
3.10.4.5	CanIDValue3.....	95
3.10.4.6	CanMBFilterMaskValue.....	96
3.10.4.7	CanTableIDType.....	97
3.10.5	CanControllerBaudrateConfig.....	97
3.10.5.1	CanControllerCheckCanStandard.....	97
3.10.5.2	CanAdvancedSetting.....	98
3.10.5.3	CanBusLength.....	98
3.10.5.4	CanPropDelayTranceiver.....	99
3.10.5.5	CanControllerPrescaller.....	99
3.10.5.6	CanControllerPrescaller_Alt.....	100
3.10.5.7	CanControllerBaudRate.....	100
3.10.5.8	CanControllerPropSeg.....	100
3.10.5.9	CanControllerSeg1.....	101
3.10.5.10	CanControllerSeg2.....	102
3.10.5.11	CanSyncJumpWidth.....	102
3.10.6	Can HardwareObject.....	103
3.10.6.1	CanHandleType.....	104
3.10.6.2	CanIdType.....	104
3.10.6.3	CanIdValue.....	105

Section number	Title	Page
3.10.6.4	CanMBPrio.....	106
3.10.6.5	CanObjectId.....	107
3.10.6.6	CanObjectType.....	107
3.10.6.7	CanControllerRef.....	108
3.10.6.8	CanFilterMaskRef.....	109
3.10.6.9	CanMainFunctionRWPeriodRef.....	109
3.10.7	Can MainFunction Read Period Parameters.....	110
3.10.7.1	CanMainFunctionReadPeriod.....	111
3.10.8	Can MainFunction Write Period Parameters.....	111
3.10.8.1	CanMainFunctionWritePeriod.....	112
3.10.9	Can Common Published Information.....	112
3.10.9.1	ArReleaseMajorVersion.....	113
3.10.9.2	ArReleaseMinorVersion.....	114
3.10.9.3	ArReleaseRevisionVersion.....	114
3.10.9.4	ModuleId.....	114
3.10.9.5	SwMajorVersion.....	115
3.10.9.6	SwMinorVersion.....	115
3.10.9.7	SwPatchVersion.....	115
3.10.9.8	VendorApiInfix.....	116
3.10.9.9	VendorId.....	116
3.11	Wakeup Configuration.....	116
3.12	Configuration Exporting.....	117
3.13	Rx Fifo.....	118
3.14	Driver Usage and Configuration Tips.....	119

Chapter 4

The Configuration of Can Bit Timing

4.1	Clock Source Description.....	121
4.2	Time Segments Calculation.....	122
4.3	CAN Bit Timing.....	124

Section number	Title	Page
	Chapter 5	
	Interrupts Implementation	
5.1	General Aspects.....	127
5.2	Interrupt Handlers.....	127



Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	15/02/2012	Navnath	Updated for MPC560xB BETA 0.9.0 Release
1.1	07/05/2013	Navnath	Updated for MPC560xB BETA 1.0.0 Release
1.2	06/06/2014	Linh Nguyen Van	1.0.1 Release



Chapter 2

Introduction

This User Manual describes Freescale Semiconductor AUTOSAR Controller Area Network (CAN) for MPC5607B.

AUTOSAR CAN driver configuration parameters and deviations from the specification are described in Can Driver chapter of this document. AUTOSAR CAN driver requirements and APIs are described in the AUTOSAR CAN driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor:

Table 2-1. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100
-------------------------	--

Table 2-1. MPC560XB Derivatives

	mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
--	--

All of the above microcontroller devices are collectively named as MPC560XB.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Micro Controller Unit

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003 CAN Driver Software Specification Document.	4.0.0 Rev 0003
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
8	Bolero 1M cut 1.0 Errata Sheet (MPC5606B_0N13E)	Rev. , 12 DEC 2013
9	Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

2.6 Symbolic Names DISCLAIMER

All containers having the symbolic name tag set as true in the Autosar schema will generate defines like:

```
#define <Container_Short_Name> <Container_ID>
```

For this reason it is forbidden to duplicate the name of such containers across the MCAL configuration, or to use names that may trigger other compile issues (e.g. match existing #ifdefs arguments).

Chapter 3

Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003 CAN Driver Software Specification document (See Table [Reference List](#)).

3.2 Driver Design Summary

The MPC560XB contains up to six Controller Area Network (FlexCAN) blocks.

Each FlexCAN module is a communication controller implementing the CAN protocol. The CAN protocol interface (CPI) sub-module manages the serial communication on the CAN bus, requesting RAM access for receiving and transmitting message frames, validating received messages and performing error handling. The message buffer management (MBM) sub-module handles message buffer selection for reception and transmission, taking care of arbitration and ID matching algorithms. The bus interface unit (BIU) sub-module controls the access to and from the internal interface bus, to establish connection to the CPU and other blocks.

The FlexCAN has these major features:

- 64 flexible message buffers (MBs) of zero to eight bytes data length.
- Individual Rx mask registers per message buffer.
- Powerful Rx FIFO ID filtering, capable of matching incoming IDs against either 8 Extended, 16 Standard, or 32 Partial (8 bits) IDs, with individual masking capability.
- ListenOnly capability.
- Programmable loop-back mode supporting self-test operation.
- Maskable interrupts.
- Low power modes.
- Unused MB and Rx Mask Register space can be used as general purpose RAM space.

Note

256k derivatives have only one Controller Area Network (FlexCAN) block and 32 message buffers (MBs). 256k derivatives have RXGMASK, RX14MASK and RX15MASK as it does not support individual Rx Mask per Message Buffer

3.3 Calling Driver APIs

1. void Can_GetVersionInfo (Std_VersionInfoType* versioninfo)

This function returns the version information of this module.

versioninfo - Pointer where to store the version information of this module.

```
typedef struct {
    uint16 vendorID;
    uint16 moduleID;
    uint8 sw_major_version;
    uint8 sw_minor_version;
    uint8 sw_patch_version;
} Std_VersionInfoType;
```

Sample code:

```
/* main_app.c */
.....
Std_VersionInfoType version;
uint16 vendor_id;

Can_Init( Can_Cfg1);
#if (CAN_VERSION_INFO_API == STD_ON)
    Can_GetVersionInfo( &version);
    vendor_id = version.vendorID;
#endif /* (CAN_VERSION_INFO_API == STD_ON) */
.....
```

Note

This API can be used only if CAN_VERSION_INFO_API is set to STD_ON.

2. void Can_Init (const Can_ConfigType* Config)

This function initializes the module.

Config - Pointer to driver configuration. This is the type of the external data structure containing the overall initialization data for the CAN driver and SFR settings affecting all controllers. Furthermore it contains pointers to controller configuration structures. The contents of the initialization data structure are CAN hardware specific.

For Pre-Compile configuration the Can_cfg.h file will contain the extern declaration of "Can_ConfigType" structure.

```
.....
#ifdef CAN_PRECOMPILE_SUPPORT
    #define CAN_INIT_CONFIG_PC_DEFINES \
        extern CONST(Can_ConfigType, CAN_CONST) Can_ConfigSet_PC;
#endif
.....
```

Can_Cfg.c file will contains the structure type:

```
.....
CONST(Can_ConfigType, CAN_CONST) Can_ConfigSet_PC = {
    .....
};
.....
```

For Post-Build configuration the Can_Cfg.h file will contain the extern declaration of "Can_ConfigType" structure.

```
.....
#ifndef CAN_PRECOMPILE_SUPPORT
    #define CAN_INIT_CONFIG_PB_DEFINES \
        extern CONST(Can_ConfigType, CAN_CONST) CanConfigSet_0_PB;
#endif
.....
```

Can_PBcfg.c file will contains the structure type:

```
.....
CONST(Can_ConfigType, CAN_CONST) CanConfigSet_0_PB = {
    .....
};
.....
```

Can.h export driver configuration:

```
#ifdef CAN_PRECOMPILE_SUPPORT
    /* Export Driver configuration */
    CAN_INIT_CONFIG_PC_DEFINES
#else
    /* Export Driver configuration */
    CAN_INIT_CONFIG_PB_DEFINES
#endif
```

Sample code:

```
/* main_app.c */
.....
#ifdef CAN_PRECOMPILE_SUPPORT
    CONST(Can_ConfigType, CAN_CONST) *Can_Cfg1 = &Can_ConfigSet_PC;
#else
```

```

        CONST(Can_ConfigType, CAN_CONST) *Can_Cfg1 = &CanConfigSet_0_PB;
    #endif
    Can_Init( Can_Cfg1);
    .....

```

3. Std_ReturnType Can_ChangeBaudrate(uint8 Controller, const uint16 Baudrate)

This function change the baudrate for the configured controller.

Controller - Can controller to be initialized - based on configuration order list (CanControllerId).

Baudrate - Baudrate for which the controller shall be initialised.

Can_Cfg.h contains the controller indexes:

```

.....
#define CanA0 0U /* Default configuration for FlexCAN_A */
.....

```

Note

This API can be used only if
CAN_CHANGE_BAUDRATE_SUPPORT is set to STD_ON.

4. Std_ReturnType Can_CheckBaudrate(uint8 Controller, const uint16 Baudrate)

This function check the controller's configured baudrate.

Controller - Can controller to be initialized - based on configuration order list (CanControllerId).CAN Controller to check for the support of a certain baudrate

Baudrate - Baudrate to check in kbps

Note

This API can be used only if
CAN_CHANGE_BAUDRATE_SUPPORT is set to STD_ON.

5. Can_ReturnType Can_SetControllerMode (uint8 Controller, Can_StateTransitionType Transition)

This function performs software triggered state transitions of the CAN controller State machine.

Controller - CAN controller for which the status shall be changed

Transition - CAN state transition: CAN_STOP / CAN_T_START / CAN_T_SLEEP / CAN_T_WAKEUP

Sample code:

```

/* main_app.c */
.....
Can_ReturnType ret_val = CAN_NOT_OK;

Can_Init( Can_Cfg1);
ret_val = Can_SetControllerMode( CanA0, CAN_T_START);
.....

```

6. void Can_DisableControllerInterrupts (uint8 Controller)

This function disables all interrupts for this CAN controller.

Controller - CAN controller for which interrupts shall be disabled.

Sample code:

```

/* main_app.c */
.....
Can_Init( Can_Cfg1);
Can_DisableControllerInterrupts( CanA0);
.....

```

7. void Can_EnableControllerInterrupts (uint8 Controller)

This function enables all allowed interrupts.

Controller - CAN controller for which interrupts shall be re-enabled.

```

/* main_app.c */
.....
Can_Init( Can_Cfg1);
Can_EnableControllerInterrupts( CanA0);
.....

```

8. Can_ReturnType Can_Write (Can_HwHandleType Hth, const Can_PduType* PduInfo)

This function transmit a message on the bus.

Hth - information which HW-transmit handle shall be used for transmit. Implicitly this is also the information about the controller to use because the Hth numbers are unique inside one hardware unit.

PduInfo - Pointer to SDU user memory, DLC and Identifier.

Can_Cfg.h will contains the MB indexes:

```

#define CTRL0_MB0 0U /* RECEIVE object of Can Controller ID = 0 */
#define CTRL0_MB1 1U /* RECEIVE object of Can Controller ID = 0 */
#define CTRL0_MB2 2U /* RECEIVE object of Can Controller ID = 0 */
#define CTRL0_MB3 3U /* TRANSMIT object of Can Controller ID = 0 */

```

```
#define CTRL0_MB4 4U /* TRANSMIT object of Can Controller ID = 0 */
#define CTRL0_MB5 5U /* TRANSMIT object of Can Controller ID = 0 */
#define CTRL1_MB0 6U /* RECEIVE object of Can Controller ID = 1 */
#define CTRL1_MB1 7U /* TRANSMIT object of Can Controller ID = 1 */
```

Sample code:

```
/* main_app.c */
.....
Can_ReturnType ret_val = CAN_NOT_OK;
Can_PduType CanMessage;
VAR(uint8, CAN_VAR) data[8U] = {0x77, 0x77, 0x77, 0x77, 0x77, 0x77, 0x77, 0x77};

Can_Init( Can_Cfg1);
CanMessage.length = 8U;
CanMessage.sdu = data;
CanMessage.id = 0x1U;

ret_val = Can_Write( CTRL0_MB3, &CanMessage);
.....
```

9. void Can_MainFunction_Write ()

This function performs the polling of TX confirmation and TX cancellation confirmation when CAN_TX_PROCESSING is set to POLLING.

Notice that:

```
/* Can.h */
#if (CAN_TXPOLL_SUPPORTED == STD_ON)
    extern FUNC (void, CAN_CODE) Can_MainFunction_Write( void );
#else
    #define Can_MainFunction_Write()
#endif
```

Sample code:

```
/* main_app.c */
.....
ret_val = Can_Write( CTRL0_MB3, &CanMessage);
Can_MainFunction_Write();
.....
```

Also, if different polling periods are configured, another polling function is enabled "Can_MainFunction_Write_0()". This function will poll only one Hardware Object, specified by CanMainFunctionWritePeriodRef

```
/* Can.h */
#ifdef CAN_MAINFUNCTION_PERIOD_WRITE_0
    extern FUNC (void, CAN_CODE) Can_MainFunction_Write_0( void);
#endif /* CAN_MAINFUNCTION_PERIOD_WRITE_0 */
```


Note

This API can be used only if CAN_TXPOLL_SUPPORTED is set to STD_ON.

10. void Can_MainFunction_Read ()

This function performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING.

Notice that:

```
/* Can.h */
#if (CAN_RXPOLL_SUPPORTED == STD_ON)
    extern FUNC (void, CAN_CODE) Can_MainFunction_Read( void );
#else
    #define Can_MainFunction_Read()
#endif
```

Also, if different polling periods are configured, another polling function is enabled "Can_MainFunction_Read_0()". This function will poll only one Hardware Object, specified by CanMainFunctionReadPeriodRef

```
/* Can.h */
#ifdef CAN_MAINFUNCTION_PERIOD_READ_0
    extern FUNC (void, CAN_CODE) Can_MainFunction_Read_0( void );
#endif /* CAN_MAINFUNCTION_PERIOD_READ_0 */
```

Note

This API can be used only if CAN_RXPOLL_SUPPORTED is set to STD_ON.

11. void Can_MainFunction_BusOff ()

This function performs the polling of bus-off events that are configured statically as 'to be polled'.

Notice that:

```
/* Can.h */
#if (CAN_BUSOFFPOLL_SUPPORTED == STD_ON)
    extern FUNC (void, CAN_CODE) Can_MainFunction_BusOff( void );
#else
    #define Can_MainFunction_BusOff()
#endif
```

Note

This API can be used only if CAN_BUSOFFPOLL_SUPPORTED is set to STD_ON.

12. void Can_MainFunction_Wakeup ()

This function performs the polling of wake-up events that are configured statically as 'to be polled'.

Notice that:

```
/* Can.h */
#if (CAN_WAKEUPPOLL_SUPPORTED == STD_ON)
    extern FUNC (void, CAN_CODE) Can_MainFunction_Wakeup( void );
#else
    #define Can_MainFunction_Wakeup()
#endif
```

Note

This API can be used only if
CAN_WAKEUPPOLL_SUPPORTED and
CAN_WAKEUP_SUPPORT are set to STD_ON.

13. void Can_MainFunction_Mode ()

This function performs the polling of CAN controller mode transitions..

Notice that:

```
/* Can.h */
extern FUNC (void, CAN_CODE) Can_MainFunction_Mode( void);
```

14. void Can_CheckWakeup (uint8 controller)

This function checks if a wakeup has occurred for the given controller.

Sample code:

```
/* main_app.c */
.....
Std_ReturnType std_ret_val = E_NOT_OK;

Can_Init( Can_Cfg1);
.....
std_ret_val = Can_CheckWakeup( CanA0);
.....
```

Note

This API can be used only if CAN_WAKEUP_SUPPORT is
set to STD_ON.

15. void Can_AbortMb (Can_HwHandleType Hth)

This function write a abort code (b'1001) to MBCS[CODE] field of the MB.

Sample code:

```
/* main_app.c */
.....
Can_PduType    CanMessage;
Can_ReturnType ret_val = CAN_NOT_OK;

Can_Init( Can_Cfg1);
ret_val = Can_SetControllerMode( CanA0, CAN_T_START);
ret_val = Can_Write( CTRL0_MB3, &CanMessage);
Can_AbortMb ( Hth);
.....
```

Note

This API can be used only if
CAN_API_ENABLE_ABORT_MB is set to STD_ON.

16. Std_ReturnType Can_SetClockMode(uint8 can_controller, Can_ClockMode can_clk_mode)

This function is configuring Can controller to run on the same baudrate, but having a different MCU source clock..

Sample code:

```
/* main_app.c */
Std_ReturnType can_return = E_NOT_OK;
Can_ReturnType ret_val = CAN_NOT_OK;
.....
Can_Init( Can_Cfg1);
can_return = Can_SetClockMode( controller, CAN_NORMAL);
.....
can_return = Can_SetClockMode( controller, CAN_ALTERNATE);
ret_val = Can_SetControllerMode( controller, CAN_T_START);
.....
```

Note

This API can be used only if CAN_DUAL_CLOCK_MODE is
set to STD_ON.

3.4 Deviation from Requirements

The driver deviates from the AUTOSAR CAN Driver software specification in some places.

There are also some additional requirements (on top of requirements detailed in AUTOSAR CAN Driver software specification) which need to be satisfied for correct operation.

1. The driver does not distinguish between “Extended” and “Mixed” MB types for receiving way: All Rx MBs configured as MIXED type will be converted to EXTENDED type. For transmission the CanIf will prepare the message ID with MSB bit set and based on this fact the Can module will send the message as STANDARD or EXTENDED type. See CANIF188 and CANIF281 requirements.
2. Priority inversion may occur even if Cancellation and Transmission multiplexing is enabled. When all message buffers in the transmission pool are full and scheduled for transmission a new call to Can_Write will cause a search which identifies the lowest priority message. If the lowest priority message has lower priority than the new message submitted to Can_Write, then cancellation of the message currently stored in the message buffer will be attempted. There is, however, a possibility that this lowest priority message might be successfully transmitted after the Can_Write has read the message buffer during its search. If a new high priority message is immediately scheduled for transmission (via preemptive call to Can_Write) the identification of the message buffer holding the lowest priority message will no longer be correct (in the underlying Can_Write which has been preempted). This may lead to the message not being cancelled (as it now may have a higher priority). In this case Can_Write will not repeat the search for the lowest priority message and priority inversion may occur (if there is another message with lower priority scheduled for transmission in a different message buffer). Whether this scenario can or cannot occur in a particular application depends on implementation of the CanIf.
3. The driver does not depend of the OS to get a timer value.
4. For functions that are blocking (need to wait a limited period of time for something to happen) the time duration is not given in seconds, but in loops.
5. There is only one present for the polling functions Can_MainFunctions_Read and Can_MainFunctions_Write, because there is only one reference of the HardwareObject to do polling.
6. The base address for the controllers is not user input.

Table [Table 3-1](#) provides Status column description.

Table 3-1. Deviations Status Column Description

Term	Definition
N/A	Not available
N/T	Not testable
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the CAN driver.

Table 3-2. CAN Deviations Table

Requirement	Status	Description	Notes
CAN011	N/S	The CAN driver directly uses the buffer of the upper layer. It is the responsibility of the upper layer to keep the buffer consistent. The function CanIf_Transmit copies the L-SDU either directly in the CAN Hardware or buffers it if CAN Hardware transmit resources are presently not available. The L-SDU source buffer (provided by source layer, e.g. COM) can be written with the next value as soon as CanIf_Transmit returns. The source layer (i.e. COM) is responsible that the L-SDU buffer is not overwritten until the function CanIf_Transmit returned. SDU must be protected by the layer that calls CanIf_Transmit.	Can module is not responsible for keeping the buffers consistent. Upper layer should provide it.
CAN323_Conf	N/I	CAN_HANDLE_TYPE Specifies the type (Full-CAN or Basic-CAN) of a hardware object.	Controller doesn't provide any bit field to differentiate BASIC-CAN and FULL-CAN.
CAN242	N/S	If an off-chip CAN controller is used, the driver uses services of other MCAL drivers (i.e. SPI). These drivers need to be up and running before the CAN controller can be initialized. The sequence of initialization of different drivers is partly specified in [7]. Only Synchronous APIs may be used because the CAN driver does not provide callback functions that can be called by the MCAL driver. Thus the type of connection between μ C and CAN Hardware Unit has only impact on implementation and not on the API.	No off-chip controller is used.
CAN110	N/S	There is no requirement regarding the execution order of the CAN main processing functions.	Application Code Requirement.
CAN240	N/S	The Can module's environment shall make sure that the Mcu module is initialized before initializing the Can module.	Can driver cannot access any variable of Mcu module for checking the state.
CAN077	N/S	For CAN Hardware Units of different type, different Can modules shall be implemented.	Current platforms have only one type of hardware unit.
CAN397	N/S	The Can module shall include the header file Os.h file.	No services of other drivers are used by Can driver.
CAN389_Conf	N/S	Specifies the CAN controller base address.	The possibility for user to input the base address of the controller is not supported by the Can Driver. The base address is given in the Base module.

3.5 Function Definitions

APIs of all functions supported by the driver are as per AUTOSAR CAN Driver software specification Version 4.0 Rev0003.

3.5.1 Function Can_Init

Initialize the CAN driver. SID = 0x00.

Prototype: void Can_Init(const Can_ConfigType *Config);

Table 3-3. Can_Init Arguments

Type	Name	Direction	Description
const Can_ConfigType *	Config	input	Pointer to driver configuration.

Return: void

Initialize all the controllers. The CAN module shall be initialized by Can_Init(<&Can_Configuration>) service call during the start-up. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Can_Init shall be called at most once during runtime.

post:: Can_Init shall initialize all the controllers and set the driver in READY state.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.2 Function Can_GetVersionInfo

Returns the version information of this module. SID = 0x07.

Prototype: void Can_GetVersionInfo(Std_VersionInfoType *versioninfo);

Table 3-4. Can_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	versioninfo	input	A pointer to location to store version info Must be omitted if the function does not have parameters.

Return: void

This routine is called by:

Note

Violates `Can_c_REF_2` Violates MISRA 2004 Advisory Rule 19.15, Repeated include file `MemMap.h`

- CanIf or an upper layer according to Autosar requirements.

pre:: The `CAN_VERSION_INFO_API` define must be configured on.

post:: The version information is return if the parameter `versionInfo` is not a null pointer.

Note

Violates `Can_h_REF_2` Violates MISRA 2004 Advisory Rule 8.10

3.5.3 Function Can_SetControllerMode

Put the controller into a required state. SID = 0x03.

Prototype: `Can_ReturnType Can_SetControllerMode(uint8 Controller, Can_StateTransitionType Transition);`

Table 3-5. Can_SetControllerMode Arguments

Type	Name	Direction	Description
uint8	Controller	input	- Can controller for which the status shall be changed - based on configuration order list (<code>CanControllerId</code>).
Can_StateTransitionType	Transition	input	- Possible transitions (<code>CAN_T_STOP</code> / <code>CAN_T_START</code> / <code>CAN_T_SLEEP</code> / <code>CAN_T_WAKEUP</code>)

Return: `Can_ReturnType` Result of the transition.

Table 3-6. Can_SetControllerMode Returns

Value	Description
<code>CAN_OK</code>	Transition initiated.
<code>CAN_NOT_OK</code>	Development or production error.

Switch the controller from one state to another. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Before changing the controller state the driver must be initialized.

post:: After the transition to the new state the interrupts required for that state must be enebaled.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.4 Function Can_DisableControllerInterrupts

Disable INTs. SID = 0x04.

Prototype: void Can_DisableControllerInterrupts(uint8 Controller);

Table 3-7. Can_DisableControllerInterrupts Arguments

Type	Name	Direction	Description
uint8	Controller	input	Can controller for which interrupts shall be disabled - based on configuration order list (CanControllerId).

Return: void

Switch OFF the controller's interrupts. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized before changing the interrupts state (en/dis).

post:: Controller must not respond to any interrupt assertion.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

Note

The maximum number of nested calls to Can_DisableControllerInterrupts is limited to 127. This function may not be preempted by code which calls function Can_InitController

3.5.5 Function Can_EnableControllerInterrupts

Enable INTs. SID = 0x05.

Prototype: void Can_EnableControllerInterrupts(uint8 Controller);

Table 3-8. Can_EnableControllerInterrupts Arguments

Type	Name	Direction	Description
uint8	Controller	input	Can controller for which interrupts shall be disabled - based on configuration order list (CanControllerId).

Return: void

Switch ON the controller's interrupts. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized before changing the interrupts state (en/dis).

post:: Controller must respond to interrupt assertion.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

Note

This function may not be preempted by code which calls function Can_InitController.

3.5.6 Function Can_Write

Transmit information on CAN bus. SID = 0x06.

Prototype: Can_ReturnType Can_Write(Can_HwHandleType Hth, const Can_PduType *PduInfo);

Table 3-9. Can_Write Arguments

Type	Name	Direction	Description
Can_HwHandleType	Hth	input	Information which HW-transmit handle shall be used for transmit. Implicitly this is also the information about the controller to use because the Hth numbers are unique inside one hardware unit.
const Can_PduType *	PduInfo	input	Pointer to SDU user memory, DLC and Identifier.

Return: Can_ReturnType Result of the write operation.

Table 3-10. Can_Write Returns

Value	Description
CAN_OK	Write command has been accepted.
CAN_NOT_OK	Development error occurred.
CAN_BUSY	No of TX hardware buffer available or preemptive call of Can_Write() that can't be implemented reentrant.

Can_Write checks if hardware transmit object that is identified by the HTH is free. Can_Write checks if another Can_Write is ongoing for the same HTH. a) hardware transmit object is free: The mutex for that HTH is set to 'signaled' the ID, DLC and SDU are put in a format appropriate for the hardware (if necessary) and copied in the

appropriate hardware registers/buffers. All necessary control operations to initiate the transmit are done. The mutex for that HTH is released. The function returns with CAN_OK. b) hardware transmit object is busy with another transmit request. The function returns with CAN_BUSY. c) A preemptive call of Can_Write has been issued, that could not be handled reentrant (i.e. a call with the same HTH). The function returns with CAN_BUSY the function is non blocking d) The hardware transmit object is busy with another transmit request for an L-PDU that has lower priority than that for the current request The transmission of the previous L-PDU is cancelled (asynchronously). The function returns with CAN_BUSY. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized and MB must be configured for Tx.

post:: The data can be transmitted or rejected because of another data with a higher priority.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.7 Function Can_CheckWakeup

Process check of WakeUp condition. SID = 0x0B.

Prototype: Std_ReturnType Can_CheckWakeup(uint8 controller);

Table 3-11. Can_CheckWakeup Arguments

Type	Name	Direction	Description
uint8	controller	input	Can controller ID - based on configuration order list (CanControllerId).

Return: Std_ReturnType Result of the wakeup verification.

Table 3-12. Can_CheckWakeup Returns

Value	Description
E_OK	Wakeup was detected for the given controller.
E_NOT_OK	No wakeup was detected for the given controller.

This service shall evaluate the WakeupSource parameter to get the information, which dedicate wakeup source needs to be checked, either a CAN transceiver or controller device. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized.

post:: Return the Wakeup event occurrence.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.8 Function Can_MainFunction_Write

Function called at fixed cyclic time. SID 0x01.

Prototype: void Can_MainFunction_Write(void);

Service for performs the polling of TX confirmation and TX cancellation confirmation when CAN_TX_PROCESSING is set to POLLING. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized.

post:: Send the data from that MB that is configured for Tx.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.9 Function Can_MainFunction_Read

Function called at fixed cyclic time, with polling on only one MessageBuffer.

Prototype: void Can_MainFunction_Read(void);

Service for performs the polling of TX confirmation and TX cancellation confirmation when CAN_TX_PROCESSING is set to POLLING. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

Service for performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING. This routine is called by:

pre:: Driver must be initialized.

post:: Send the data from that MB that is configured for Tx.

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized.

post:: Receive the data from that MB that is configured for Rx.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

Note

This function may not be preempted by code which calls any of the driver functions.

3.5.10 Function Can_MainFunction_BusOff

Function called at fixed cyclic time, with polling on only one MessageBuffer.

Prototype: void Can_MainFunction_BusOff(void);

Service for performs the polling of RX indications when CAN_RX_PROCESSING is set to POLLING. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

Service for performs the polling of BusOff events that are configured statically as 'to be polled'. This routine is called by:

pre:: Driver must be initialized.

post:: Receive the data from that MB that is configured for Rx.

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized.

post:: Handle the Busoff event.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.11 Function Can_MainFunction_Wakeup

Function called at fixed cyclic time. SID = 0x0A.

Prototype: void Can_MainFunction_Wakeup(void);

Service for performs performs the polling of WakeUp events that are configured statically as 'to be polled'. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized.

post:: Handle the Wakeup event.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.12 Function Can_MainFunction_Mode

Function called at fixed cyclic time. SID = 0x0C.

Prototype: void Can_MainFunction_Mode(void);

Service for performs performs the polling of CAN status register flags to detect transition of CAN Controller state This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized.

post:: Handle the transition of Can Controller state.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.13 Function Can_AbortMb

Process a message buffer abort.

Prototype: void Can_AbortMb(Can_HwHandleType Hth);

Table 3-13. Can_AbortMb Arguments

Type	Name	Direction	Description
Can_HwHandleType	Hth	input	- HW-transmit handler

This function write a abort code (b'1001) to MBCS[CODE] field of the MB. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized and the current MB transmission should be ready for transmit.

Note

Not AUTOSAR required. This is user implementation. Violates
 Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.14 Function Can_SetClockMode

Process a transition from one clock source to another.

Prototype: Std_ReturnType Can_SetClockMode(uint8 can_controller, Can_ClockMode can_clk_mode);

Table 3-14. Can_SetClockMode Arguments

Type	Name	Direction	Description
uint8	can_controller	input	controller ID
Can_ClockMode	can_clk_mode	input	clock mode selection

Return: Std_ReturnType Result of the clock switch operation.

Table 3-15. Can_SetClockMode Returns

Value	Description
E_OK	Switch clock operation was ok.
E_NOT_OK	Switch clock operation was not ok.

This function is configuring Can controllers to run on the same baudrate, but having a different MCU source clock. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Driver must be initialized and all the controllers must be in Stop state.

Note

Not AUTOSAR required. This is user implementation. Violates
 Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.15 Function Can_ChangeBaudrate

Initialize the CAN controllers. SID = 0x0d.

Prototype: Std_ReturnType Can_ChangeBaudrate(uint8 Controller, const uint16 Baudrate)

Table 3-16. Can_ChangeBaudrate Arguments

Type	Name	Direction	Description
uint8	Controller	input	CAN Controller, whose baudrate shall be changed
const uint16	Baudrate	input	Requested baudrate in kbps

Return: Std_ReturnType

Initialize all the controllers. Initialize the controller based on ID input parameter. This routine is called by:

- CanIf or an upper layer according to Autosar requirements.

pre:: Before controller re-initialization the driver must be initialized and the controllers must be in Stop state.

post:: Interrupts and MBs must be configured for respond to CAN bus.

Note

Violates Can_h_REF_2 Violates MISRA 2004 Advisory Rule 8.10

3.5.16 Function Can_CheckBaudrate

Initialize the CAN controllers. SID = 0x0e.

Prototype: Std_ReturnType Can_CheckBaudrate(uint8 Controller, const uint16 Baudrate)

Table 3-17. Can_CheckBaudrate Arguments

Type	Name	Direction	Description
uint8	Controller	input	CAN Controller to check for the support of a certain baudrate
const uint16	Baudrate	input	Baudrate to check in kbps

Return: Std_ReturnType

The service Can_CheckBaudrate(Controller, Baudrate) shall be called by CanIf_CheckBaudrate() for the requested CAN controller.

- CanIf or an upper layer according to Autosar requirements.

Note

: If Can supports changing of the baudrate and thus this service, shall be configurable via CAN_CHANGE_BAUDRATE_SUPPORT.

3.5.17 Function Can_GetErrorStatus

Returns the value of Error Status Register. SID = 0x11.

Prototype: `uint32 Can_GetErrorStatus(uint8 can_controller)`

Table 3-18. Can_GetErrorStatus Arguments

Type	Name	Direction	Description
uint8	Controller	input	CAN Controller

Return: uint32

This function checks the error status and intimated to upper layer

pre:: Driver must be initialized and the current MB transmission should be ready for transmit.

Note

Violates `Can_h_REF_2` Violates MISRA 2004 Advisory Rule 8.10

Limitation: Status register contents change time to time, storing the status registers in a local variable and then proceeding with operations based on the stored variable might lead to undesired results.

3.6 Structure Definitions

Data structures supported by the driver are as per AUTOSAR CAN Driver software specification Version 4.0 Rev0003.

3.6.1 Structure Can_ConfigType

Top Level structure containing all Driver configuration.

A pointer to this structure is transmitted to `Can_Init()` to initialize the driver at startup. The application selects one of the configurations by using a pointer to one of the elements of this array as a parameter of the `Can_Init` function.

Declaration

```
typedef struct
{
    const Can_PCallOutType Can_LPduReceiveCalloutFunction,
    uint8 Can_ControllersConfigured,
```



```

Can_IdPtrType FilterMasksPtr,
const Can_ControllerBaudrateConfigType *const ControllerConfigsPtr,
const Can_MBConfigContainerType MBConfigContainer,
const Can_ControllerDescriptorType * ControllerDescriptors,
    const Can_RxFifoTableIdConfigType *Can_RxFifoTableIdConfigPtr,
    uint32 Can_max_object_Id,
const uint8 Can_ControllerID_Mapping[CAN_MAX_OBJECT_ID],
    const Can_ObjType Can_ObjectType_Mapping[CAN_MAX_OBJECT_ID]
} Can_ConfigType;

```

Table 3-19. Structure Can_ConfigType member description

Member	Description
Can_LPduReceiveCalloutFunction	Pointer to LPDU receive callout function.
Can_ControllersConfigured	Number of Can Controllers configured in configuration.
FilterMasksPtr	Pointer to the first FilterMask value - any controller can have many filter masks for Can messages.
MBConfigContainer	Pointer to the first MB configuration of this Controller.
ControllerDescriptors	Pointer to the first FlexCAN Controller description.
Can_RxFifoTableIdConfigPtr	Pointer to the Table IDs for the RxFifo.
Can_max_object_Id	Maximum Object IDs configured
Can_ControllerID_Mapping	Controller ID mapping
Can_ObjectType_Mapping	Can Object Type mapping

3.6.2 Structure Can_ControllerDescriptorType

Structure for describing individual FlexCAN controllers on the chip.

HRH = Hardware Receive Handle (HRH) is defined and provided by the CAN driver. Each HRH represents exactly one hardware object. The HRH can be used to optimize software filtering. HTH = The Hardware Transmit Handle (HTH) is defined and provided by the CAN driver. Each HTH represents one or several hardware objects, that are configured as hardware transmit pool.

Declaration

```

typedef struct
{
    const uint8 ControllerOffset,
    const uint8 MaxMBCount,
    const Can_HwHandleType HthOffset,
        const uint8 MaxBaudRateCount,
        const uint8 DefaultBaudRateIndex,
        const Can_ControllerBaudrateConfigType
*ControllerBaudrateConfigsPtr,
    const uint32 CanWakeupSourceID,
    const uint8 RxFifoTableIdConfigIndex,
    const Can_PCallbackType Can_RxFifoOverflowNotification,
    const Can_PCallbackType Can_RxFifoWarningNotification,
    const Can_PCallbackType Can_ErrorNotification
        const uint32 Options,
    const uint8 Can_NumberOfMB,

```

```

        const Can_HwHandleType FirstHrhIndex
    } Can_ControllerDescriptorType;

```

Table 3-20. Structure Can_ControllerDescriptorType member description

Member	Description
ControllerOffset	Hardware Offset for Can controller: FLEXCAN_A = Offset[0], FLEXCAN_B = Offset[1], ...
MaxMBCount	Max Message Buffer number.
MaxBaudRateCount	Max BaudRate number.
DefaultBaudRateIndex	Default baudrate index.
ControllerBaudrateConfigsPtr	Pointer to the Configuration of Baudrate timing parameter for FlexCAN baudrate controller (CTRL value register).
CanWakeupSourceID	WakeUp source ID (type def uint32 EcuM_WakeupSourceType - in EcuM.h (uint32)).
RxFifoTableIDConfigIndex	RxFifo TableID Controller index.
Can_RxFifoOverflowNotification	Pointer to RX FIFO Overflow notification function.
Can_RxFifoWarningNotification	Pointer to RX FIFO Warning notification function.
Can_ErrorNotification	Pointer to Error interrupt notification function (ESR[ERR_INT]).
Options	BusOff Sw Recovery, RXFifo En, IDAM Type, Event Trigger Mode TxProcessing/RxProcessing/BusoffProcessing/WakeUpProcessing: Polling vs Interrupt mode
Can_NumberOfMB	Number of message Buffers available for FlexCan unit.
FirstHrhIndex	Index for First Hrh.

3.6.3 Structure Can_ControllerBaudrateConfigType

Configuration of FlexCAN controller.

This structure is initialized by Tresos considering user settings. Used by Can_ConfigType. Passed to Can_InitController() at initialization.

Declaration

```

typedef struct
{
    const uint32 ControlRegister,
    const uint32 ControlRegister_Alternate,
    const uint32 Options,
    const uint8 Can_NumberOfMB
} Can_ControllerBaudrateConfigType;

```

Table 3-21. Structure Can_ControllerBaudrateConfigType member description

Member	Description
ControlRegister	Content of the Control Register (CTRL) fields: PRES DIV, RJW, PSEG1, PSEG2, CLKSRC, LPB, SMP, BOFF_REC, LOM, PROPSEG.

Table continues on the next page...

Table 3-21. Structure Can_ControllerBaudrateConfigType member description (continued)

Member	Description
ControlRegister_Alternate	Content of the Control Register (CTRL) fields: PRES DIV, RJW, PSEG1, PSEG2, CLK SRC, LPB, SMP, BOFF_REC, LOM, PROPSEG.
Options	BusOff Sw Recovery, RXFifo En, IDAM Type, Event Trigger Mode TxProcessing/RxProcessing/BusoffProcessing/WakeUpProcessing: Polling vs Interrupt mode.
Can_NumberOfMB	Number of message Buffers available for FlexCan unit.

3.6.4 Structure Can_ControllerStatusType

Records the status of a CAN Controller during runtime.

Records the status of a CAN Controller during runtime.

Note

This structure is not configured by Tresos.

Declaration

```
typedef struct
{
    uint32 TxGuard[2],
    uint32 MBInterruptMask[2],
    PduIdType TxPduId[64],
    sint8 IntDisableLevel,
    Can_HwHandleType CancelMBIndex,
    uint8 FirstTxMBIndex,
    uint8 ControllerSWFlag,
    uint8 InterruptMode,
    Can_ControllerStateType ControllerState,
    uint32 Can_MBMapping[CAN_MAXMB_SUPPORTED],
    uint8 CurrentBaudRateIndex
} Can_ControllerStatusType;
```

Table 3-22. Structure Can_ControllerStatusType member description

Member	Description
TxGuard	Guard bits for EXCLUSIVE ACCESS to Tx MBs.
MBInterruptMask	Pre-calculated MB INT masks (used in Can_EnableControllerInterrupts).
TxPduId	Storage space for PDU_ID (supplied in call to Can_Write and needed after Tx in CanIf_TxConfirmation).
IntDisableLevel	Storage space for Can_DisableControllerInterrupts nesting level.
CancelMBIndex	Index of MB buffer being cancelled.
FirstTxMBIndex	Index of the first MB used for Tx for a specific controller. This value is relative to 0 (which is first MB).
ControllerSWFlag	Software flags for tracking interrupt changes state.
InterruptMode	Global interrupt authorization state.
ControllerState	FlexCAN controller power state.

Table continues on the next page...

Table 3-22. Structure Can_ControllerStatusType member description (continued)

Member	Description
Can_MBMapping	Map for every MB the HOH assigned according to configuration.
CurrentBaudRateIndex	Current controller baudrate

3.6.5 Structure Can_MBConfigContainerType

Type for storing Message Buffer configurations.

The MessageBufferConfigs array is sorted according to:

- HRHs first, HTHs next (AutoSAR requirement)
- Controller ID (HRHs and HTHs belonging to all controllers must be grouped together)
- Message ID (to ensure top priority IDs are first which means they will be serviced first)

Declaration

```
typedef struct
{
    const Can_MBConfigObjectType *const MessageBufferConfigsPtr,
        const Can_HwHandleType MessageBufferConfigCount
    } Can_MBConfigContainerType;
```

Table 3-23. Structure Can_MBConfigContainerType member description

Member	Description
MessageBufferConfigCount	Number of elements in the array -(having 6 controllers with 64MBs each uint8 is not enough to store this value -> the type is extended to uint16).
MessageBufferConfigsPtr	Pointer to the MB array .

3.6.6 Structure Can_MBConfigObjectType

Can_MBConfigObjectType.

Type for storing information about Message Buffers (CAN hardware objs). Used by Can_MBConfigContainerType.

Declaration

```
typedef struct
{
    Can_HwHandleType IdMaskIndex,
    const uint8 ControllerId,
    CanIdType IdType,
```

```

Can_ObjType MBType,
Can_IdType MessageId,
const uint8 LocalPriority,
uint32 HWObjID,
const uint8 CanMainFuncRWPeriodRef
} Can_MBConfigObjectType;

```

Table 3-24. Structure Can_MBConfigObjectType member description

Member	Description
IdMaskIndex	Index into array of Can_FilterMaskType values (uint8/uint16), Current MB and the corresponding filter mask.
ControllerId	Controller ID (index into controller address array containing Can_ControllerPtrType).
IdType	ID type: CAN_EXTENDED, CAN_STANDARD, CAN_MIXED.
MBType	Receive/Transmit.
MessageId	(extended identifier) (uint16/uint32). configurable by CanHardwareObject/ CanIdValue.
LocalPriority	Local priority bits used for arbitration.
HWObjID	HW Object ID.
CanMainFuncRWPeriodRef	read write period reference.
LocalPriority	Local priority bits used for arbitration.
HWObjID	HW Object ID.
CanMainFuncRWPeriodRef	Read Write Period Reference .

3.6.7 Structure Can_PduType

Can_PduType.

Type used to provide ID, DLC, SDU from CAN interface to CAN driver. HTH/HRH = ID+DLC+SDU.

Declaration

```

typedef struct
{
    Can_IdType id,
    PduIdType swPduHandle,
    uint8 length,
    uint8 * sdu
} Can_PduType;

```

Table 3-25. Structure Can_PduType member description

Member	Description
id	CAN L-PDU = Data Link Layer Protocol Data Unit. Consists of Identifier, DLC and Data(SDU) It is uint32 for CAN_EXTENDEDID=STD_ON, else is uint16.

Table continues on the next page...

Table 3-25. Structure Can_PduType member description (continued)

Member	Description
swPduHandle	The L-PDU Handle = defined and placed inside the CanIf module layer. Each handle represents an L-PDU, which is a constant structure with information for Tx/Rx processing.
length	DLC = Data Length Code (part of L-PDU that describes the SDU length).
sdu	CAN L-SDU = Link Layer Service Data Unit. Data that is transported inside the L-PDU.

3.6.8 Structure Can_RxFiFoTableIdConfigType

Rx Fifo Table IDs and Filter Masks.

Rx Fifo Table IDs and Filter Masks

Declaration

```
typedef struct
{
    const uint32 TableId,
        const uint32 TableFilterMask,
    } Can_RxFiFoTableIdConfigType;
```

Table 3-26. Structure Can_RxFiFoTableIdConfigType member description

Member	Description
TableFilterMask	Table with the Filter masks.
TableId	Table with the IDs specific for Rx Fifo.

3.7 Define Definitions

Data defines supported by the driver are as per AUTOSAR CAN Driver software specification Version 4.0 Rev0003.

3.7.1 Define CAN_API_ENABLE_ABORT_MB

Support for Special MB Abort API.

Definition: `#define CAN_API_ENABLE_ABORT_MB (STD_ON)`

3.7.2 Define CAN_BCC_SUPPORT_ENABLE

Defines if Backwards Compatibility Configuration (BCC) feature of CAN controller is used in the configuration. If BCC feature of CAN controller is enabled, Individual Rx masking and queue feature are disabled. If BCC feature of CAN controller is disabled, Individual Rx masking and queue feature are enabled.

Definition: `#define CAN_BCC_SUPPORT_ENABLE (STD_OFF)`

3.7.3 Define CAN_BUSOFFPOLL_SUPPORTED

This macro enables `Can_MainFunction_BusOff()` if at least one controller is set to process BusOff in Polling Mode.

Definition: `#define CAN_BUSOFFPOLL_SUPPORTED (STD_ON)`

3.7.4 Define CAN_DEV_ERROR_DETECT

Switches the Development Error Detection and Notification ON or OFF.

Definition: `#define CAN_DEV_ERROR_DETECT (STD_ON)`

3.7.5 Define CAN_DUAL_CLOCK_MODE

Enable Non-Autosar API for Dual-Clock support.

Definition: `#define CAN_DUAL_CLOCK_MODE (STD_OFF)`

3.7.6 Define CAN_E_DEFAULT

Development errors.

Definition: `#define CAN_E_DEFAULT (uint8)0x08U`

3.7.7 Define CAN_E_DATA_LOST

Development errors.

Definition: `#define CAN_E_DATA_LOST (uint8)0x07U`

3.7.8 Define CAN_E_PARAM_CONTROLLER

Development errors.

Definition: `#define CAN_E_PARAM_CONTROLLER (uint8)0x04U`

3.7.9 Define CAN_E_PARAM_DLC

Development errors.

Definition: `#define CAN_E_PARAM_DLC (uint8)0x03U`

3.7.10 Define CAN_E_PARAM_HANDLE

Development errors.

Definition: `#define CAN_E_PARAM_HANDLE (uint8)0x02U`

3.7.11 Define CAN_E_PARAM_POINTER

Development errors.

Definition: `#define CAN_E_PARAM_POINTER (uint8)0x01U`

3.7.12 Define CAN_E_TRANSITION

Development errors.

Definition: `#define CAN_E_TRANSITION (uint8)0x06U`

3.7.13 Define CAN_E_UNINIT

Development errors.

Definition: `#define CAN_E_UNINIT (uint8)0x05U`

3.7.14 Define CAN_E_PARAM_BAUDRATE

Development errors.

Definition: `#define CAN_E_PARAM_BAUDRATE (uint8)0x09U`

3.7.15 Define CAN_ERROR_NOTIFICATION_ENABLE

Error notification enabled/disabled.

Definition: `#define CAN_ERROR_NOTIFICATION_ENABLE (STD_ON)`

3.7.16 Define CAN_EXTENDEDID

Extended identifiers.

Definition: `#define CAN_EXTENDEDID (STD_ON)`

3.7.17 Define CAN_HW_TRANSMIT_CANCELLATION

Support for Transmission Cancellation.

Definition: `#define CAN_HW_TRANSMIT_CANCELLATION (STD_ON)`

3.7.18 Define CAN_IDENTICAL_ID_CANCELLATION

Support for Identical Id Cancellation.

Definition: `#define CAN_IDENTICAL_ID_CANCELLATION (STD_ON)`

3.7.19 Define CAN_INIT_CONFIG_CONTROLLERS_PC

Definition: `#define CAN_INIT_CONFIG_CONTROLLERS_PC extern
CONST(Can_ControllerBaudrateConfigType, CAN_CONST)
ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT_0];`

3.7.20 Define CAN_INIT_CONFIG_PC_DEFINES

Export Can_ConfigType structure.

Definition: `#define CAN_INIT_CONFIG_PC_DEFINES extern CONST(Can_ConfigType, CAN_CONST)
Can_ConfigSet_PC;`

3.7.21 Define CAN_ISROPTCODESIZE

Optimization of interrupt service code for size.

Definition: `#define CAN_ISROPTCODESIZE (STD_OFF)`

3.7.22 Define CAN_MAINFUNCTION_MODE_PERIOD

Periods for cyclic call of Main function Mode.

Definition: `#define CAN_MAINFUNCTION_MODE_PERIOD 0U`

3.7.23 Define CAN_MAINFUNCTION_PERIOD_BUSOFF

Periods for cyclic call of Main function.

Definition: `#define CAN_MAINFUNCTION_PERIOD_BUSOFF 0U`

3.7.24 Define CAN_MAINFUNCTION_PERIOD_READ

Periods for cyclic call of Main function.

Definition: `#define CAN_MAINFUNCTION_PERIOD_READ 1U`

3.7.25 Define CAN_MAINFUNCTION_PERIOD_WAKEUP

Periods for cyclic call of Main function.

Definition: `#define CAN_MAINFUNCTION_PERIOD_WAKEUP 0U`

3.7.26 Define CAN_MAINFUNCTION_PERIOD_WRITE

Periods for cyclic call of Main function Write.

Definition: `#define CAN_MAINFUNCTION_PERIOD_WRITE 3U`

3.7.27 Define CAN_MAXCTRL_SUPPORTED

Maximum possible controllers per specific derivative.

Definition: `#define CAN_MAXCTRL_SUPPORTED 2U`

3.7.28 Define CAN_MAXMB_SUPPORTED

Maximum possible Message Buffers per controller specific to this platform.

Definition: `#define CAN_MAXMB_SUPPORTED 32U`

3.7.29 Define CAN_MBCOUNTENSION

Extended number of can hardware objects.

Definition: `#define CAN_MBCOUNTENSION (STD_ON)`

3.7.30 Define CAN_MIX_MB_SUPPORT

Platform support mix of controllers with 64 and 32 MBs.

Definition: `#define CAN_MIX_MB_SUPPORT (STD_OFF)`

3.7.31 Define CAN_MULTIPLEXED_TRANSMISSION

Support for Multiplexed Transmission.

Definition: `#define CAN_MULTIPLEXED_TRANSMISSION (STD_ON)`

3.7.32 Define CAN_PRECOMPILE_SUPPORT

Precompile Support On.

Definition: `#define CAN_PRECOMPILE_SUPPORT`

3.7.33 Define CAN_RXFIFO_ENABLE

Support for Rx Fifo.

Definition: `#define CAN_RXFIFO_ENABLE (STD_ON)`

3.7.34 Define CAN_RXFIFO_EVENT_UNIFIED

Set if Rx Fifo events (Warning/Overflow/FrameAvailable) are configured on the same int on INTC vector table.

Definition: `#define CAN_RXFIFO_EVENT_UNIFIED (STD_ON)`

3.7.35 Define CAN_RXPOLL_SUPPORTED

This macro enables `Can_MainFunction_Read()` if at least one controller is set to process Rx in Polling Mode.

Definition: `#define CAN_RXPOLL_SUPPORTED (STD_ON)`

3.7.36 Define CAN_SID_ABORT_MB

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_ABORT_MB (uint8)0x10U`

3.7.37 Define CAN_SID_CBK_CHECK_WAKEUP

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_CBK_CHECK_WAKEUP (uint8)0x0BU`

3.7.38 Define CAN_SID_DISABLE_CONTROLLER_INTERRUPTS

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_DISABLE_CONTROLLER_INTERRUPTS (uint8)0x04U`

3.7.39 Define CAN_SID_ENABLE_CONTROLLER_INTERRUPTS

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_ENABLE_CONTROLLER_INTERRUPTS (uint8)0x05U`

3.7.40 Define CAN_SID_GET_VERSION_INFO

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_GET_VERSION_INFO (uint8)0x07U`

3.7.41 Define CAN_SID_INIT

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_INIT (uint8)0x00U`

3.7.42 Define CAN_SID_INIT_CONTROLLER

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_INIT_CONTROLLER (uint8)0x02U`

3.7.43 Define CAN_SID_MAIN_FUNCTION_BUS_OFF

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_MAIN_FUNCTION_BUS_OFF (uint8)0x09U`

3.7.44 Define CAN_SID_MAIN_FUNCTION_MODE

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_MAIN_FUNCTION_MODE (uint8)0x0CU`

3.7.45 Define CAN_SID_MAIN_FUNCTION_READ

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_MAIN_FUNCTION_READ (uint8)0x08U`

3.7.46 Define CAN_SID_MAIN_FUNCTION_WAKEUP

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_MAIN_FUNCTION_WAKEUP (uint8)0x0AU`

3.7.47 Define CAN_SID_MAIN_FUNCTION_WRITE

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_MAIN_FUNCTION_WRITE (uint8)0x01U`

3.7.48 Define CAN_SID_SET_CONTROLLER_MODE

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_SET_CONTROLLER_MODE (uint8)0x03U`

3.7.49 Define CAN_SID_SETCLOCKMODE

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_SETCLOCKMODE (uint8)0x0FU`

3.7.50 Define CAN_SID_WRITE

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_WRITE (uint8)0x06U`

3.7.51 Define CAN_SID_CHANGE_BAUDRATE

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_CHANGE_BAUDRATE (uint8)0x0DU`

3.7.52 Define CAN_SID_CHECK_BAUDRATE

Service ID (APIs) for Det reporting.

Definition: `#define CAN_SID_CHECK_BAUDRATE (uint8)0x0EU`

3.7.53 Define CAN_TIMEOUT_DURATION

(CAN113_Conf) Specifies the maximum time for blocking function until a timeout is detected. Unit in loops.

Definition: `#define CAN_TIMEOUT_DURATION 20U`

3.7.54 Define CAN_TXPOLL_SUPPORTED

This macro enables `Can_MainFunction_Write()` if at least one controller is set to process Tx in Polling Mode.

Definition: `#define CAN_TXPOLL_SUPPORTED (STD_ON)`

3.7.55 Define CAN_VERSION_INFO_API

Support for version info API.

Definition: `#define CAN_VERSION_INFO_API (STD_ON)`

3.7.56 Define CAN_WAKEUP_SUPPORT

CAN driver support for wakeup over CAN Bus.

Definition: `#define CAN_WAKEUP_SUPPORT (STD_OFF)`

3.7.57 Define CAN_WAKEUPPOLL_SUPPORTED

This macro enables `Can_MainFunction_Wakeup()` if at least one controller is set to process WakeUp in Polling Mode.

Definition: `#define CAN_WAKEUPPOLL_SUPPORTED (STD_ON)`

3.7.58 Define Can_Abort_Only_One_MB

Support for MB Abort API.

When this parameter is true only one MB will be aborted by Can_AbortMb()

When this parameter is false all the associated mailboxes will be aborted when multiplex transmission is enabled

Definition: `#define Can_Abort_Only_One_MB (STD_ON)`

3.7.59 Define CAN_MAXCTRL_CONFIGURED

Maximum possible controllers configured.

Definition: `#define CAN_MAXCTRL_CONFIGURED 2U`

3.7.60 Define CAN_MAXMB_CONFIGURED

Maximum Message Buffers configured

Definition: `#define CAN_MAXMB_CONFIGURED 17U`

3.8 Enum Definitions

Data Enums supported by the driver are as per AUTOSAR CAN Driver software specification Version 4.0 Rev0003.

3.8.1 Enumeration Can_ClockMode

Can_ClockMode.

CAN source clock selection used in Can_SetClockMode Non-Autosar API.

Table 3-27. Enumeration Can_ClockMode Values

Value	Description
CAN_NORMAL = 0U	Standard configuration (default).
CAN_ALTERNATE	Second configuration (special).

3.8.2 Enumeration Can_ControllerStateType

CAN Controller states.

States that defines the controllers.

Table 3-28. Enumeration Can_ControllerStateType Values

Value	Description
CAN_STOPPED = 0U	Controller in state STOPPED.
CAN_STARTED	Controller in state STARTED.
CAN_SLEEPS	Controller in state SLEEPED.

3.8.3 Enumeration Can_ObjType

Can_ObjType.

Used for value received by Tressos interface configuration. Describe the MB configuration.

Table 3-29. Enumeration Can_ObjType Values

Value	Description
CAN_RECEIVE = 0U	Receive MB.
CAN_TRANSMIT	Transmit MB.

3.8.4 Enumeration Can_ReturnType

Can_ReturnType.

CAN Return Types from Functions.

Table 3-30. Enumeration Can_ReturnType Values

Value	Description
CAN_OK = 0U	Operation was ok executed.
CAN_NOT_OK	Operation was not ok executed.
CAN_BUSY	Operation was rejected because of busy state.

3.8.5 Enumeration Can_StateTransitionType

CAN State Modes of operation.

State transitions that are used by the function CAN_SetControllerMode().

Table 3-31. Enumeration Can_StateTransitionType Values

Value	Description
CAN_T_STOP = 0U	CANIF_CS_STARTED -> CANIF_CS_STOPPED.
CAN_T_START	CANIF_CS_STOPPED -> CANIF_CS_STARTED.
CAN_T_SLEEP	CANIF_CS_STOPPED -> CANIF_CS_SLEEP.
CAN_T_WAKEUP	CANIF_CS_SLEEP -> CANIF_CS_STOPPED.

3.8.6 Enumeration Can_StatusType

Can_StatusType.

CAN Driver status used for checking and preventing double driver initialization.

CAN_UNINIT = The CAN controller is not initialized. The CAN Controller is not participating on the CAN bus. All registers belonging to the CAN module are in reset state, CAN interrupts are disabled. CAN_READY = Controller has initialized: static variables, including flags; Common setting for the complete CAN HW unit; CAN controller specific settings for each CAN controller.

Table 3-32. Enumeration Can_StatusType Values

Value	Description
CAN_UNINIT = 0U	Driver not initialized.
CAN_READY	Driver ready.

3.8.7 Enumeration CanIdType

CanIdType.

Used for value received by Tressos interface configuration. Used to differentiate Extended, Mixed or Standard Id type.

Table 3-33. Enumeration CanIdType Values

Value	Description
CAN_EXTENDED = 0U	Extended ID (29 bits).
CAN_STANDARD	Standard ID (11 bits).
CAN_MIXED	Mixed ID (29 bits).

3.9 Typedef Definitions

Data Typedefs supported by the driver are as per AUTOSAR CAN Driver software specification Version 4.0 Rev0003.

3.9.1 Typedef Can_HwHandleType

Can_HwHandleType.

Type: uint32

Represents the hardware object handles of a CAN hardware unit. For CAN hardware units with more than 255 HW objects use extended range.

- used by "Can_Write" function. The driver does not distinguish between Extended and Mixed transmission modes. Extended transmission mode of operation behaves the same as Mixed mode.

3.9.2 Typedef Can_IdPtrType

Can_IdPtrType.

Type: const uint32 *const

Type for storing pointer to the Identifier Length Type.

- used by "Can_ConfigType" structure (pointer to the FilterMasks).

3.9.3 Typedef Can_IdType

Can_IdType.

Type: uint32

Type for storing the Identifier Length Type: Normal /Extended.

- used by "Can_MessageBufferConfigObjectType" structure. The driver does not distinguish between Extended and Mixed transmission modes. Extended transmission mode of operation behaves the same as Mixed mode.

3.9.4 Typedef Can_PCallbackType

Type for pointer to function.

Type: void(*)

Type for pointer to function. Used for user handlers from plugin.

3.9.5 Typedef Can_PtrControlerDescriptorType

Type: const Can_ControlerDescriptorType *

3.9.6 Typedef Can_PtrMBConfigContainerType

Type: const Can_MBConfigContainerType *

3.10 Configuration Parameters

As per the AUTOSAR specification the driver has two types of configurations parameters: **Pre-Compile** parameters and **Post-Build** parameters. Pre-Compile parameters are stored in the file Can_Cfg.h and Can_Cfg.c. Post-Build parameters are stored in the file Can_PBcfg.c.

The files to be used for different configuration types are listed below:

1. **Variant PC:** Can_Cfg.h, Can_Cfg.c
2. **Variant PB:** Can_Cfg.h, Can_PBcfg.c
3. **Variant LT:** Not Applicable

A section for **Can_PBcfg.c** file is needed in linker file to place the post build configuration in desired location.

Please refer to section-8 of **AUTOSAR_SWS_C_ImplementationRules** file for complete details on configuration types.

3.10.1 Can-General Parameters

CanGeneral parameters, their possible values and meaning are described in the following text. CanGeneral parameters are implemented as preprocessor defines.

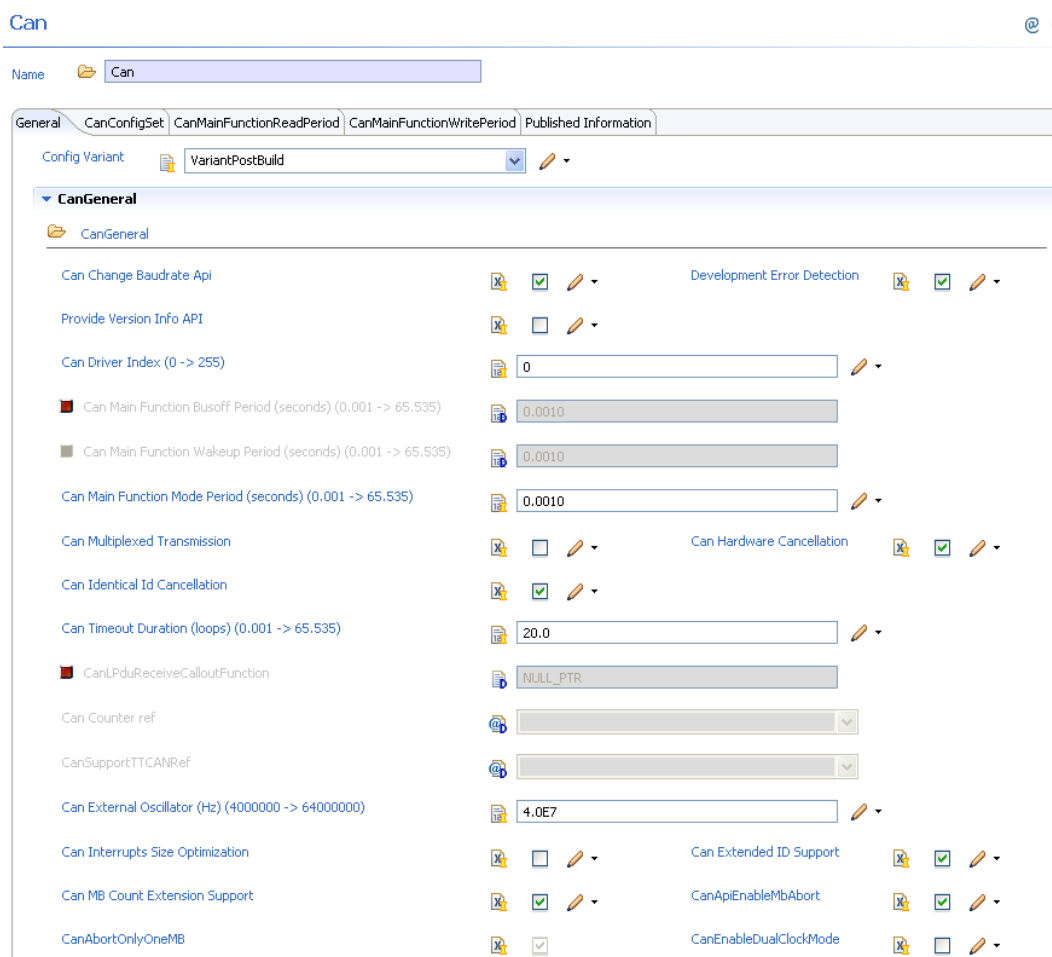


Figure 3-1. Can General Parameters

3.10.1.1 IMPLEMENTATION_CONFIG_VARIANT

Table 3-34. IMPLEMENTATION_CONFIG_VARIANT

Description	Defines whether pre-compile version is used. Using this option with VariantPostBuild value, Tressos can generate many CanConfigSet variants.
Class	Implementation Specific Parameter
Range	VariantPreCompile, VariantPostBuild
Default	VariantPreCompile
Source File	Can_Cfg.h

Table continues on the next page...

**Table 3-34. IMPLEMENTATION_CONFIG_VARIANT
(continued)**

Source Representation	#define CAN_PRECOMPILE_SUPPORT
Autosar 4.0 Requirement	NA

Note

This parameter permit to generate many configurations if VariantPostBuild is selected.

3.10.1.2 CanDevErrorDetection**Table 3-35. CanDevErrorDetection**

Description	Switches the Development Error Detection and Notification ON or OFF.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define CAN_DEV_ERROR_DETECT STD_OFF / STD_ON
Autosar 4.0 Requirement	CAN064_Conf

Note

Setting this control to false will generate code that is reduced, but some Autosar requirements are not tested (no Det errors are reported in this way).

3.10.1.3 CanVersionInfoApi**Table 3-36. CanVersionInfoApi**

Description	Defines whether version information reporting should be included at compile time (STD_ON) or excluded (STD_OFF).
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define CAN_VERSION_INFO_API STD_OFF / STD_ON
Autosar 4.0 Requirement	CAN106_Conf

Note

Setting this define to STD_OFF will decrease the code size, but not support for Can_GetVersionInfo API will be available.

3.10.1.4 CanIndex**Table 3-37. CanIndex**

Description	Specifies the Instance ID of this module instance. If only one instance is present it shall have the ID 0.
Class	Autosar Parameter
Range	Integer
Default	0
Source File	Can_Cfg.h
Source Representation	#define CAN_INSTANCE 0
Autosar 4.0 Requirement	CAN320_Conf
NOTE	<i>This parameter is transmitted as the second parameter to the Det_Reporterror function.</i>

Note

This parameter is transmitted as the second parameter to the Det_Reporterror function.

3.10.1.5 CanMainFunctionBusOffPeriod**Table 3-38. CanMainFunctionBusOffPeriod**

Description	Describes the period for cyclic call to Can_MainFunction_Busoff (in seconds).
Class	Autosar Parameter
Range	0.001 .. 65.535
Default	0.001
Source File	Can_Cfg.h
Source Representation	#define CAN_MAINFUNCTION_PERIOD_BUSOFF 0.001U
Autosar 4.0 Requirement	CAN355_Conf

Note

This parameter is optional. The period value is not used in the Can driver. It should be exported to SchM for using it when polling mode is selected.

3.10.1.6 CanMainFunctionModePeriod

Table 3-39. CanMainFunctionModePeriod

Description	Describes the period for cyclic call to Can_MainFunction_Mode (in seconds).
Class	Autosar Parameter
Range	0.001 .. 65.535
Default	0.001
Source File	Can_Cfg.h
Source Representation	#define CAN_MAINFUNCTION_MODE_PERIOD 0.001U
Autosar 4.0 Requirement	CAN376_Conf

Note

This period value is not used in the Can driver. It should be exported to SchM for using it when polling mode is selected.

3.10.1.7 CanMainFunctionWakeupPeriod

Table 3-40. CanMainFunctionWakeupPeriod

Description	Describes the period for cyclic call to Can_MainFunction_Wakeup (in seconds).
Class	Autosar Parameter
Range	0.001 .. 65.535
Default	0.001
Source File	Can_Cfg.h
Source Representation	#define CAN_MAINFUNCTION_PERIOD_WAKEUP 0.001U
Autosar 4.0 Requirement	CAN357_Conf

Note

This parameter is optional. The period value is not used in the Can driver. It should be exported to SchM for using it when polling mode is selected.

3.10.1.8 CanIdenticalIdCancellation

Table 3-41. CanIdenticalIdCancellation

Description	Specifies if identical ID cancellation shall be supported ON or OFF.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h

Table continues on the next page...

Table 3-41. CanIdenticalIdCancellation (continued)

Source Representation	#define CAN_IDENTICAL_ID_CANCELLATION STD_ON
Autosar 4.0 Requirement	CAN378_Conf

Note

Setting this control to false, the Can Module shall not initiate a cancellation, when the hardware transmit object assigned by a HTH is busy, an L-PDU with identical priority is requested to be transmitted.

3.10.1.9 CanMultiplexedTransmission**Table 3-42. CanMultiplexedTransmission**

Description	Defines whether support for multiplex transmission should be included at compile time (STD_ON) or excluded (STD_OFF).
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define CAN_MULTIPLEXED_TRANSMISSION STD_OFF / STD_ON
Autosar 4.0 Requirement	CAN095_Conf

Note

When this define is set to STD_ON the multiplex transmission is used. Multiplex transmission means to send a Can message from any MB that is free, MB that has CanObjectId equal to the one transmitted as parameter to Can_Write. Multiple MBs can have the same ObjectID.

3.10.1.10 CanHardwareCancellation**Table 3-43. CanHardwareCancellation**

Description	Specifies if hardware cancellation shall be supported ON or OFF.
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define CAN_HW_TRANSMIT_CANCELLATION STD_OFF / STD_ON
Autosar 4.0 Requirement	CAN069_Conf

Note

Setting this control to false will generate code that is reduced, but no support for MB cancellation is available.

3.10.1.11 CanTimeoutDurationFactor**Table 3-44. CanTimeoutDurationFactor**

Description	Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops.
Class	Autosar Parameter
Range	Integer
Default	20000
Source File	Can_Cfg.h
Source Representation	#define CAN_TIMEOUT_DURATION [!"num:i(CanTimeoutDuration)"!]U
Autosar 4.0 Requirement	CAN113_Conf

Note

This value represents a number of finite "while-loops" that the Driver can wait until a hardware set is configured. There is no correspondence between this number of loops and the number of uC cycles.

3.10.1.12 CanCrystalFrequencyHz**Table 3-45. CanCrystalFrequencyHz**

Description	Defines the crystal frequency used.
Class	Implementation Specific Parameter
Range	4 MHz-64 MHz
Default	40 MHz
Source File	NA
Source Representation	NA
Autosar 4.0 Requirement	NA

Note

This parameter should have the same value as the board external oscillator. This value can be the clock source for Can controller if "CanClockFromBus" = false.

3.10.1.13 CanLPduReceiveCalloutFunction

Table 3-46. CanLPduReceiveCalloutFunction

Description	Specifies the name of the callout function.
Class	Autosar Parameter
Range	NA
Default	NULL_PTR
Source File	Can_Cfg.h
Source Representation	extern FUNC(boolean, COM_APPL_CODE) 'LPduCalloutFunction_name'(uint8 Hrh, Can_IdType CanId, uint8 CanDlc, const uint8 *CanSduPtr);
Autosar 4.0 Requirement	CAN434_Conf

Note

This parameter defines the existence and the name of a callout function that is called after a successful reception of a received CAN Rx L-PDU. If this parameter is omitted no callout shall take place.

3.10.1.14 CanCodeSizeOptimization

Table 3-47. CanCodeSizeOptimization

Description	Enables optimization of interrupt service routines for code size.
Class	Implementation Specific Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define CAN_ISROPTCODESIZE STD_OFF / STD_ON
Autosar 4.0 Requirement	NA

Note

This parameter permit to implement all interrupts routines for all MBs with the same routine. Code size is reduced. Ex: Can_IsrFCA_MB_00_03, Can_IsrFCA_MB_04_07, etc, are replaced by Can_IsrFCA_UNI.

3.10.1.15 CanExtendedIdSupport

Table 3-48. CanExtendedIdSupport

Description	Enables support of Extended/Mixed mode.
Class	Implementation Specific Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define CAN_EXTENDEDID STD_OFF / STD_ON
Autosar 4.0 Requirement	NA

Note

This parameter permit to enable the Message ID representation on uint32 size variable, else uint16 is used.

3.10.1.16 CanMBCountExtensionSupport

Table 3-49. CanMBCountExtensionSupport

Description	Enables support of more than 255 Can Hardware Objects.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.h
Source Representation	#define CAN_MBCOUNTTEXTENSION STD_OFF / STD_ON
Autosar 4.0 Requirement	NA

Note

This parameter permit to declare more than 255 MBs in the configuration. If total MBs for all controllers exceed 255 then this parameter must be on.

3.10.1.17 CanApiEnableMbAbort

Table 3-50. CanApiEnableMbAbort

Description	Enables an additional API, to write an ABORT code (b1001) to the MBCB filed of the MB to abort a message transmission.
Class	Implementation Specific Parameter
Range	True, False
Default	False

Table continues on the next page...

Table 3-50. CanApiEnableMbAbort (continued)

Source File	Can_Cfg.h
Source Representation	#define CAN_API_ENABLE_ABORT_MB STD_OFF / STD_ON
Autosar 4.0 Requirement	NA

Note

This parameter value is considered only if "CanHardwareCancellation" is true.

3.10.1.18 CanEnableDualClockMode**Table 3-51. CanEnableDualClockMode**

Description	Enables support for dual clock API. Can controller is able to run on the same baudrate over CAN bus using 2 different source clocks that can be changed.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.h
Source Representation	#define CAN_DUAL_CLOCK_MODE STD_OFF / STD_ON
Autosar 4.0 Requirement	NA

Note

This parameter is visible and configurable only if Can.CanConfig.DualClockMode=STD_ON from Resource files.

3.10.1.19 CanCounterRef**Table 3-52. CanCounterRef**

Description	Contains a reference to the counter.
Class	Autosar Parameter
Range	NA
Default	Reference to [OsCounter]
Source File	NA
Source Representation	NA
Autosar 4.0 Requirement	CAN431_Conf

Note

This parameter contains a reference to the counter, which is used by the CAN driver. The Can Driver does not support this requirement.

3.10.1.20 CanMainFunctionReadPeriodRef**Table 3-53. CanMainFunctionReadPeriodRef**

Description	Reference to CAN Hardware Object which shall be polled with the configured CanMainFunctionReadPeriod
Class	Autosar Parameter
Range	NA
Default	Reference to [CanHardwareObject]
Source File	Can_Cfg.h
Source Representation	#define CAN_MAIN_FUNCTION_READ_PERIOD_REF (uint32)1U
Autosar 4.0 Requirement	CAN432_Conf

Note

This reference shall only be configurable if more than one period is configured via CanMainFunctionReadPeriod.

3.10.1.21 CanMainFunctionWritePeriodRef**Table 3-54. CanMainFunctionWritePeriodRef**

Description	Reference to CAN Hardware Object which shall be polled with the configured CanMainFunctionWritePeriod
Class	Autosar Parameter
Range	NA
Default	Reference to [CanHardwareObject]
Source File	Can_Cfg.h
Source Representation	#define CAN_MAIN_FUNCTION_WRITE_PERIOD_REF (uint32)1U
Autosar 4.0 Requirement	CAN433_Conf

Note

This reference shall only be configurable if more than one period is configured via CanMainFunctionWritePeriod.

3.10.1.22 CanSupportTTCANRef

Table 3-55. CanSupportTTCANRef

Description	Refers to CanIfSupportTTCAN parameter in the CAN Interface Module configuration.
Class	Autosar Parameter
Range	NA
Default	NA
Source File	NA
Source Representation	NA
Autosar 4.0 Requirement	CAN430_Conf

Note

The CanIfSupportTTCAN parameter defines whether TTCAN is supported. This requirement is not supported by Can Driver.

3.10.1.23 CanMainFunctionRWPeriods

Table 3-56. CanMainFunctionRWPeriods

Container Name	CanMainFunctionRWPeriods{CAN_MAIN_FUNCTION_RWPERIODS}
Description	Reference to CAN Controller to which the HOH is associated to.
Class	Autosar Parameter
Autosar 4.0 Requirement	CAN437_Conf

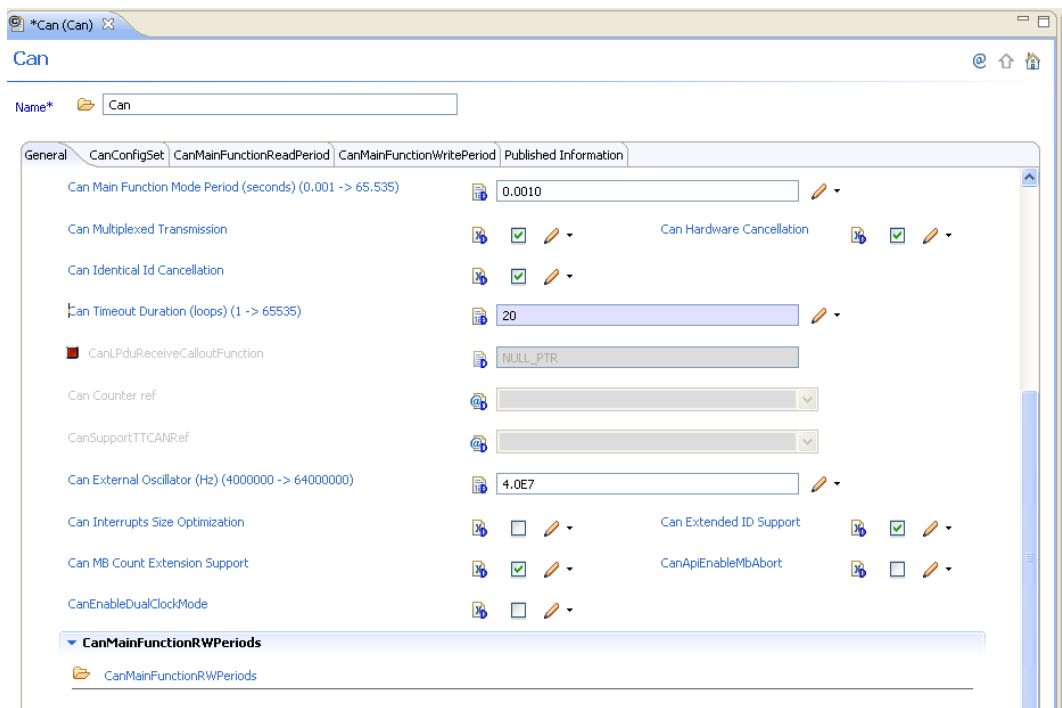


Figure 3-2. CanMainFunctionRWPeriods

Note

This parameter describes the period for cyclic call to Can_MainFunction_Write and Can_MainFunction_Read . Unit is seconds. Different poll-cycles will be configurable if more than one CanMainFunctionWritePeriod or Can_MainFunction_ReadPeriod are configured. In this case multiple Can_MainFunction_Write() or Can_MainFunction_Read() will be provided by the CAN Driver module.

3.10.1.24 Can_ChangeBaudrate

Table 3-57. Can_ChangeBaudrate

Description	Defines whether baudrate information reporting should be included at compile time (STD_ON) or excluded (STD_OFF).
Class	Autosar Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define CAN_CHANGE_CHECK_BAUDRATE STD_OFF / STD_ON
Autosar 4.0 Requirement	CAN436_Conf

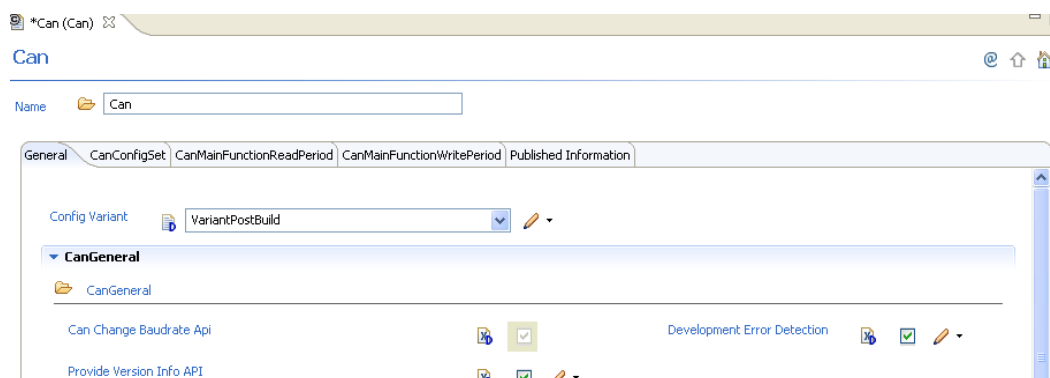


Figure 3-3. can_changebaudrate

Note

This parameter should be enabled always to initialise all the configured FlexCAN controller after the call of Can_Init() autosar function. Hence the default for this parameter is always true and non editable.

3.10.1.25 CanAbortOnlyOneMB

Table 3-58. CanAbortOnlyOneMB

Description	When this parameter is true only one MB will be aborted by Can_AbortMb() and When this parameter is false all the associated mailboxes will be aborted when multiplex transmission is enabled.
Class	Implementation Specific Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define Can_Abort_Only_One_MB STD_ON
Autosar 4.0 Requirement	NA

3.10.2 Can-Controller Parameters

Can-Controller parameters, their possible values and meaning are described in the following text. The Can-Controller parameters are implemented as constant structures and arrays stored in flash memory of the MCU.

CanController

Name  CanController_0

General

CanControllerBaudRate

CanFilterMask

CanRxFifoTable

Can Hardware Channel

 FlexCAN_A 

Can Controller Activation

 ☒ 

Can Controller Base Address (0 -> 4294967295)

 0

Can Controller ID (0 -> 255)

 0 

Can Rx Processing Type

 Interrupt 

Can Tx Processing Type

 Interrupt 

Can BusOff Processing Type

 Polling 

Can Wakeup Processing Type

 Polling 

Can BCC Enable

 ☐ 

Can Listen Only Mode (CTRL[LOM])

 ☐ 

Can Software BusOff Recovery

 ☐ 

Can Triple Sampling Mode Enable (CTRL[SMP])

 ☐ 

Can Local Priority Enable (MCR[LPRIO_EN])

 ☐ 

Can Clock from Bus (CTRL[CLK_SRC])

 ☐ 

Can CPU Reference Clock

Can CPU Reference Clock Alternate

Can RxFifo Enable

 ☐ 

CanRxFifoWarningNotification

 NULL_PTR

CanRxFifoOverflowNotification

 NULL_PTR

Can Error Notification Enable

 ☒ 

CanErrorControllerNotification

 CanErrorController_Ctrl0_Notif 

 CanWakeupSourceRef

 /EcuM/EcuM/EcuMConfiguration_0/EcuMCommonConfigur 

Can RxFifo

 Can RxFifo

Can RxFifo ID Acceptance Mode

 A 

Figure 3-4. Can Controller Parameters

3.10.2.1 CanHwChannel

Table 3-59. CanHwChannel

Description	Specifies which one of the on-chip FlexCan interface is associated with the controller ID.
Class	Implementation Specific Parameter

Table continues on the next page...

Table 3-59. CanHwChannel (continued)

Range	FlexCAN_A, FlexCAN_B, ...
Default	FlexCAN_A
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerDescriptorType, CAN_CONST) ControllerDescriptors_PC[CAN_MAXCONTROLLERCOUNT] = { /* Can Controller Offset on chip */ FLEXCAN_A_OFFSET, } </pre>
Autosar 4.0 Requirement	NA

Note

This parameter set the Can hardware channel for which the settings are implemented. The order from hardware (Can_A first and Can_B second) is not mandatory to be respected in the CanController list from Tresos plugin.

3.10.2.2 CanControllerActivation**Table 3-60. CanControllerActivation**

Description	Defines if a CAN controller is used in the configuration.
Class	Autosar Parameter
Range	True , False
Default	True
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerDescriptorType, CAN_CONST) ControllerDescriptors_PC[CAN_MAXCONTROLLERCOUNT] = { /* ControllerState */ CONTRL_ENABLED, } </pre>
Autosar 4.0 Requirement	CAN315_Conf

Note

This parameter permit to use the current settings for one of Can controllers. If this control is set to 'false' no Can controller initialization is made (NULL_PTR).

3.10.2.3 CanControllerBaseAddress

Table 3-61. CanControllerBaseAddress

Description	Specifies the CAN controller base address.
Class	Autosar Parameter
Range	0 .. 4294967295
Default	0
Source File	NA
Source Representation	NA
Autosar 4.0 Requirement	CAN382_Conf

Note

This requirement is not supported by the Can Driver.

3.10.2.4 CanControllerId

Table 3-62. CanControllerId

Description	This parameter provides the controller ID which is unique in a given CAN Driver. The value for this parameter starts with 0 and continues without any gaps.
Class	Autosar Parameter
Range	0 .. 255
Default	0
Source File	Can_Cfg.h
Source Representation	#define CanController_0 0U /* Default configuration for FlexCAN_A */
Autosar 4.0 Requirement	CAN316_Conf

Note

This parameter set an ID for the current Can controller configuration. Also a define is generated for each Can controller.

3.10.2.5 CanWakeupSupport

Table 3-63. CanWakeupSupport

Description	Defines whether Can driver supports for Wakeup over Can bus.
Class	Autosar Parameter
Range	True, False
Default	True

Table continues on the next page...

Table 3-63. CanWakeupSupport (continued)

Source File	Can_Cfg.h
Source Representation	#define CAN_WAKEUP_SUPPORT STD_OF / STD_ON
Autosar 4.0 Requirement	CAN330_Conf

Note

This parameter enables the wakeup functionality in case the platform supports this feature. It is related to internal wakeup.

3.10.2.6 CanRxProcessing**Table 3-64. CanRxProcessing**

Description	Specifies if RX events are polled inside Can_MainFunction_Read or cause an interrupt.
Class	Autosar Parameter
Range	Polling , Interrupt
Default	Polling
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Controller Options ===== */ /* RxPoll Enabled */ CAN_CONTROLLERCONFIG_RXPOL_EN } </pre>
Autosar 4.0 Requirement	CAN317_Conf

Note

This parameter set how it is implemented the handling of Rx confirmation events: by polling or by interrupt.

3.10.2.7 CanTxProcessing**Table 3-65. CanTxProcessing**

Description	Specifies if TX events are polled inside Can_MainFunction_Write or cause an interrupt.
Class	Autosar Parameter
Range	Polling , Interrupt
Default	Polling

Table continues on the next page...

Table 3-65. CanTxProcessing (continued)

Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Controller Options ===== */ /* TxPoll Enabled */ CAN_CONTROLLERCONFIG_TXPOL_EN I } </pre>
Autosar 4.0 Requirement	CAN318_Conf

Note

This parameter set how it is implemented the handling of Tx confirmation events: by polling or by interrupt.

3.10.2.8 CanBusOffProcessing**Table 3-66. CanBusOffProcessing**

Description	Specifies if bus-off events are polled inside Can_Main_Function_BusOff or cause an interrupt.
Class	Autosar Parameter
Range	Polling , Interrupt
Default	Polling
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Controller Options ===== */ /* BusOffPoll Enabled */ CAN_CONTROLLERCONFIG_BOPOLE_N I } </pre>
Autosar 4.0 Requirement	CAN314_Conf

Note

This parameter set how it is implemented the handling of BusOff confirmation events: by polling or by interrupt.

3.10.2.9 CanWakeupProcessing

Table 3-67. CanWakeupProcessing

Description	Specifies if wakeup events are polled inside Can_Main_Function_Wakeup or cause an interrupt.
Class	Autosar Parameter
Range	Polling , Interrupt
Default	Polling
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Controller Options ===== */ /* WakeupPoll Enabled */ CAN_CONTROLLERCONFIG_WKPOL_EN } </pre>
Autosar 4.0 Requirement	CAN319_Conf

Note

This parameter set how it is implemented the handling of Wakeup confirmation events: by polling or by interrupt.

3.10.2.10 CanListenOnlyMode

Table 3-68. CanListenOnlyMode

Description	Enables the Listen only mode.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[LOM] - Listen only mode */ } </pre>
Autosar 4.0 Requirement	NA

Note

In this mode, transmission is disabled, all error counters are frozen and the module operates in a CAN Error Passive mode.

3.10.2.11 CanLoopBackMode**Table 3-69. CanLoopBackMode**

Description	Enables the Loop Back Mode.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[LPB] - Loop-back mode */ } </pre>
Autosar 4.0 Requirement	NA

Note

The bit stream output of the transmitter is fed back internally to the receiver input. The Rx CAN input pin is ignored and the Tx CAN output goes to the recessive state (logic "1").

3.10.2.12 CanSoftwareBusOffRecovery**Table 3-70. CanSoftwareBusOffRecovery**

Description	Enables Automatic Bus Recovery Off.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Controller Options ===== */ /* Software BusOff Recovery */ CAN_CONTROLLERCONFIG_BUSOFFSWREC </pre>

Table continues on the next page...

**Table 3-70. CanSoftwareBusOffRecovery
(continued)**

	 }
Autosar 4.0 Requirement	NA

Note

Enables Software Bus Off recovery when automatic recovering from BusOff state is disabled for CAN controller.

3.10.2.13 CanAutoBusOffRecovery**Table 3-71. CanAutoBusOffRecovery**

Description	Enables Automatic Bus Recovery Off.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[BOFF_REC] - Bus off recovery */ } </pre>
Autosar 4.0 Requirement	NA

Note

Enables automatic BusOff recovery (CTRL[BOFF_REC] bit). If this bit is negated, automatic recovering from Bus Off state occurs according to the CAN Specification 2.0B. If the bit is asserted, automatic recovering from Bus Off is disabled and the module remains in Bus Off state until the bit is negated by the user.

3.10.2.14 CanTripleSamplingEnable

Table 3-72. CanTripleSamplingEnable

Description	Enables acquisition of 3 samples and majority voting for the value of received bit.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[SMP] - Sampling mode */ } </pre>
Autosar 4.0 Requirement	NA

Note

This bit defines the sampling mode of CAN bits at the Rx input.
 1 - Enables acquisition of 3 samples and majority voting for the value of received bit. 0 - Just one sample is used to determine the bit value.

3.10.2.15 CanWakeUpSourceFilter

Table 3-73. CanWakeUpSourceFilter

Description	Defines whether the integrated low-pass filter is applied to protect the Rx Can input from spurious wake up.
Class	Implementation Specific Parameter
Range	True, False
Default	True
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Controller Options ===== */ /* This setting defines whether the integrated low-pass filter is applied to protect the Rx CAN input from spurious wake up. */ CAN_CONTROLLERCONFIG_WAKEUP_SRC } </pre>
Autosar 4.0 Requirement	NA

Note

Defines whether the integrated low-pass filter is applied to protect the Rx CAN input from spurious wake up. 1 = FlexCAN uses the filtered Rx input to detect recessive to dominant edges on the CAN bus. 0 = FlexCAN uses the unfiltered Rx input to detect recessive to dominant edges on the CAN bus. This option will provide the setting for init the controller with MCR[WAK_SRC] bit set. This parameter is available only if the hardware supports wakeup source filter.

3.10.2.16 CanLowestBuffTransmitFirst**Table 3-74. CanLowestBuffTransmitFirst**

Description	This parameter defines the ordering mechanism for MB transmission.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[LBUF] - Lowest Buffer Transmitted First */ } </pre>
Autosar 4.0 Requirement	NA

Note

CTRL[LBUF]. This bit defines the ordering mechanism for Message Buffer transmission. When asserted, the MCR[LPRIO_EN] bit doesn't affect the priority arbitration.

3.10.2.17 CanLocalPriorityEn**Table 3-75. CanLocalPriorityEn**

Description	This field is used when MCR[LPRIO_EN] is set and makes sense only for Tx MBs.
Class	Implementation Specific Parameter
Range	True, False
Default	False

Table continues on the next page...

**Table 3-75. CanLocalPriorityEn
(continued)**

Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Controller Options ===== */ /* Local Priority Feature */ CAN_CONTROLLERCONFIG_LPRIO_EN } </pre>
Autosar 4.0 Requirement	NA

Note

MCR[LPRIO_EN]. This bit controls whether the local priority feature is enabled or not.

3.10.2.18 CanWarningEnable**Table 3-76. CanWarningEnable**

Description	This parameter defines if warning interrupt is enabled for Rx and Tx.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs0_PB[CAN_MAXCONTROLLERCOUNT] = { / * Warning Interrupt Enable Feature */ CAN_CONTROLLERCONFIG_WRNINT_EN </pre>
Autosar 4.0 Requirement	NA

3.10.2.19 CanClockFromBus**Table 3-77. CanClockFromBus**

Description	Switches the source clock for the module to the system bus (rather than crystal).
Class	Implementation Specific Parameter
Range	True, False

Table continues on the next page...

Table 3-77. CanClockFromBus (continued)

Default	False
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[CLK_SRC] - Clock source */ } </pre>
Autosar 4.0 Requirement	NA

Note

Switches the source clock for the module to the system bus (rather than crystal). 1 = The CAN engine clock source is the bus clock.(from MCU). 0 = The CAN engine clock source is the oscillator clock.

3.10.2.20 CanCpuClockRef**Table 3-78. CanCpuClockRef**

Description	Reference to the CPU clock configuration, which is set in the MCU driver configuration. This parameter is enabled only if "CanClockFromBus" is set to true.
Class	Autosar Parameter
Range	NA
Default	/Mcu/Mcu/McuModuleConfiguratio_0/McuClockSettingConfig/McuClockReferencePoin_0
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	NA
Autosar 4.0 Requirement	CAN313_Conf

Note

Reference to the CPU clock configuration, which is set in the MCU driver configuration. This control is used only if "CanClockFromBus" = "true". MCU plugin need to be added and then give the reference to it.

3.10.2.21 CanControllerRXFifoEnable

Table 3-79. CanControllerRXFifoEnable

Description	Defines if RX FIFO feature of CAN controller is used in the configuration.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.h
Source Representation	#define CAN_A_FIFO_EN STD_ON / STD_OFF
Autosar 4.0 Requirement	NA

Note

Defines if RX FIFO feature of CAN controller is used in the configuration. If FIFO feature of CAN controller is enabled, First 8 Message Buffers will be used by FIFO engine.

3.10.2.22 CanRxFifoWarningNotification

Table 3-80. CanRxFifoWarningNotification

Description	Defines the handler for Rx Fifo warning.
Class	Implementation Specific Parameter
Range	String
Default	NULL_PTR
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	extern FUNC(void, CAN_CODE) handler_name (void);
Autosar 4.0 Requirement	NA

3.10.2.23 CanRxFifoOverflowNotification

Table 3-81. CanRxFifoOverflowNotification

Description	Defines the handler for Rx Fifo overflow.
Class	Implementation Specific Parameter
Range	String
Default	NULL_PTR
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	extern FUNC(void, CAN_CODE) handler_name (void);
Autosar 4.0 Requirement	NA

3.10.2.24 CanErrorControllerNotification

Table 3-82. CanErrorControllerNotification

Description	Defines the handler for error controller.
Class	Implementation Specific Parameter
Range	String
Default	NULL_PTR
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	extern FUNC(void, CAN_CODE) handler_name (void);
Autosar 4.0 Requirement	NA

3.10.2.25 CanWakeupSourceRef

Table 3-83. CanWakeupSourceRef

Description	This parameter contains a reference to the Wakeup source for this controller as defined in the ECU State Manager.
Class	Autosar Parameter
Range	NA
Default	/EcuM/EcuM/EcuMConfiguration_0/EcuMWakeupSource_0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_ControllerDescriptorType, CAN_CONST) ControllerDescriptors_PC[CAN_MAXCONTROLLERCOUNT] = { /* WakeUp source ID (ECU State Manager) */ 0U, } </pre>
Autosar 4.0 Requirement	CAN359_Conf

Note

This parameter contains a reference to the Wakeup Source for this controller as defined in the ECU State Manager. Type: reference to EcuM_WakeupSourceType EcuM plugin need to be added and then give the reference to it.

3.10.2.26 CanCpuClockRef_Alternate

Table 3-84. CanCpuClockRef_Alternate

Description	Reference to the CPU clock configuration, which is set in the MCU driver configuration. This parameter is enabled only if "CanClockFromBus" is set to true.
Class	Implementation Specific Parameter
Range	NA
Default	/Mcu/Mcu/McuModuleConfiguratio_0/McuClockSettingConfig/McuClockReferencePoin_0
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	NA
Autosar 4.0 Requirement	NA

Note

Reference to the CPU clock configuration, which is set in the MCU driver configuration. This parameter is available/editable only if "Can.CanConfig.DualClockMode" is set to STD_ON from Resource files, "CanEnableDualClockMode" is set to true and "CanClockFromBus" = "true". MCU plugin need to be added and then give the reference to it.

3.10.2.27 CanBccSupport

Table 3-85. CanBccSupport

Description	Defines if Backwards Compatibility Configuration (BCC) feature of CAN controller is used in the configuration.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.h
Source Representation	#define CAN_BCC_SUPPORT_ENABLE STD_ON / STD_OFF
Autosar 4.0 Requirement	NA

Note

If BCC feature of CAN controller is enabled, Individual Rx masking and queue feature are disabled. Else, Individual Rx masking and queue feature are enabled.

3.10.2.28 CanErrorControllerNotifEn

Table 3-86. CanErrorControllerNotifEn

Description	Enables/Disables the Error Controller Notification. If Disabled, no error interrupt or notification shall take place.
Class	Implementation Specific Parameter
Range	True, False
Default	True
Source File	Can_Cfg.h
Source Representation	#define CAN_ERROR_NOTIFICATION_ENABLE STD_ON / STD_OFF
Autosar 4.0 Requirement	NA

Note

Not AutoSar Required.

3.10.2.29 CanControllerDefaultBaudrate

Table 3-87. CanControllerDefaultBaudrate

Description	Reference to baudrate configuration container configured for the Can Controller.
Class	Implementation Specific Parameter
Range	NA
Default	/Can/Can/CanConfigSet_0/CanController_0/CanControllerBaudrateConfig_0
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	NA
Autosar 4.0 Requirement	CAN435_Conf

3.10.3 CanFilterMask



Figure 3-5. CanFilterMask

3.10.3.1 CanFilterMaskValue

Table 3-88. CanFilterMaskValue

Description	Reference to the filter mask that is used for hardware filtering together with the CAN_ID_VALUE
Class	Autosar Parameter
Range	0 - 4294967296
Default	2047
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre>CONST(Can_IdType, CAN_CONST) Can_FilterMasks_PC[CAN_MAXFILTERCOUNT_0] = { /* FilterMasks_PC[0], "CanFilterMask_0" */ 0x7ffU, }</pre>
Autosar 4.0 Requirement	CAN066_Conf

Note

Describes a mask for hardware-based filtering of CAN identifiers.

3.10.4 Can RxFifo



Figure 3-6. Can RxFifo

Name  CanRxFifoTable_0

General

Can RxFifo Identifier 0	 255
Can RxFifo Identifier 1	 255
Can RxFifo Identifier 2	 255
Can RxFifo Identifier 3	 255
Can RxFifo Frame Accepted Mode	 Standard
Can RxFifo Filter Mask	 2047

Figure 3-7. Can CanRxFifoTable - CanControllerRxFifoEnable = FALSE

General

Can RxFifo Identifier 0	 255 
Can RxFifo Identifier 1	 255
Can RxFifo Identifier 2	 255
Can RxFifo Identifier 3	 255
Can RxFifo Frame Accepted Mode	 Standard 
Can RxFifo Filter Mask	 2047 

Figure 3-8. Can CanRxFifoTable - CanControllerRxFifoEnable = TRUE and CanControllerIDAcceptanceMode = A

Parameter	Value
Can RxFifo Identifier 0	255
Can RxFifo Identifier 1	255
Can RxFifo Identifier 2	255
Can RxFifo Identifier 3	255
Can RxFifo Frame Accepted Mode	Standard
Can RxFifo Filter Mask	2047

Figure 3-9. Can CanRxFifoTable - CanControllerRxFifoEnable = TRUE and CanControllerIDAcceptanceMode = B

Parameter	Value
Can RxFifo Identifier 0	255
Can RxFifo Identifier 1	255
Can RxFifo Identifier 2	255
Can RxFifo Identifier 3	255
Can RxFifo Frame Accepted Mode	Standard
Can RxFifo Filter Mask	2047

Figure 3-10. Can CanRxFifoTable - CanControllerRxFifoEnable = TRUE and CanControllerIDAcceptanceMode = C

Parameter	Value
Can RxFifo Identifier 0	255
Can RxFifo Identifier 1	255
Can RxFifo Identifier 2	255
Can RxFifo Identifier 3	255
Can RxFifo Frame Accepted Mode	Standard
Can RxFifo Filter Mask	2047

Figure 3-11. Can CanRxFifoTable - CanControllerRxFifoEnable = TRUE and CanControllerIDAcceptanceMode = D

3.10.4.1 CanControllerIDAcceptanceMode

Table 3-89. CanControllerIDAcceptanceMode

Description	This 2-bit field identifies the format of the elements of the Rx FIFO filter table.
Class	Implementation Specific Parameter
Range	A - D
Default	A
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Controller Options ===== */ /* ID Acceptance Mode A */ CAN_CONTROLLERCONFIG_IDAM_A } </pre>
Autosar 4.0 Requirement	NA

3.10.4.2 CanIDValue0

Table 3-90. CanIDValue0

Description	Specifies an ID to be used as acceptance criteria for the ID Table0. Value for this parameter should be entered based on the CanControllerIDAcceptanceMode selected as explained below: Format A - One full ID (standard or extended) per filter element Format B - One full standard ID if the CanTableIDType is Standard or one 14 most significant bit value of Extended ID if the CanTableIDType is Extended Format C - One 8 most significant bit value of Standard or Extended ID.
Class	Implementation Specific Parameter
Range	0 - 2047 for Format A and B– Standard CanTableIDType typ 0 – 536870911 for Format A – Extended CanTableIDType type 0 – 1683 for Format B – Extended CanTableIDType type 0 – 255 for Format C
Default	0xF0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> Format C CONST(Can_RxFifoTableIdConfigType, CAN_CONST) RxFifoTableID_PCCfg[CAN_MAXTABLEID_0] = { {0x11121314, /* CanRxFifoTable_0 of formatC for FlexCAN_A */ 0xffffffff }} </pre>

Table continues on the next page...

Table 3-90. CanIDValue0 (continued)

	Format B <pre>CONST(Can_RxFifoTableIdConfigType, CAN_CONST) RxFifoTableID_PBCfg [CAN_MAXTABLEID]= { { 0x1ff807f8, /* CanRxFifoTable_0 of type Standard and formatB for FlexCAN_A */ 0xffff8fff8 }}</pre> Format A <pre>CONST(Can_RxFifoTableIdConfigType, CAN_CONST) RxFifoTableID_PBCfg [CAN_MAXTABLEID]= { {0x4001ffe, /* CanRxFifoTable_3 of type Extended and formatA for FlexCAN_A */ 0xffffffe }}</pre>
Autosar 4.0 Requirement	NA

Note

Specifies an ID to be used as acceptance criteria for the ID Table 0.

3.10.4.3 CanIDValue1**Table 3-91. CanIDValue1**

Description	Specifies an ID to be used as acceptance criteria for the ID Table1. Value for this parameter should be entered based on the CanControllerIDAcceptanceMode selected as explained below: Format B - One full standard ID if the CanTableIDType is Standard or one 14 most significant bit value of Extended ID if the CanTableIDType is Extended Format C - One 8 most significant bit value of Standard or Extended ID.
Class	Implementation Specific Parameter
Range	0 - 2047 for Format B– Standard CanTableIDType type 0 – 1683 for Format B – Extended CanTableIDType type 0 – 255 for Format C
Default	0xF0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	Format C <pre>CONST(Can_RxFifoTableIdConfigType, CAN_CONST) RxFifoTableID_PCCfg[CAN_MAXTABLEID_0] = { {0x11121314, /* CanRxFifoTable_0 of formatC for FlexCAN_A */ 0xffffffff }}</pre> Format B <pre>CONST(Can_RxFifoTableIdConfigType, CAN_CONST) RxFifoTableID_PBCfg [CAN_MAXTABLEID]= {</pre>

Table continues on the next page...

Table 3-91. CanIDValue1 (continued)

	{ {0x1ff807f8, /* CanRxFifoTable_0 of type Standard and formatB for FlexCAN_A */ 0xffff8fff8 }}
Autosar 4.0 Requirement	NA

Note

Specifies an ID to be used as acceptance criteria for the ID Table 1.

3.10.4.4 CanIDValue2**Table 3-92. CanIDValue2**

Description	Specifies an ID to be used as acceptance criteria for the ID Table2. Value for this parameter should be entered based on the CanControllerIDAcceptanceMode selected as explained below C- One 8 most significant bit value of Standard or Extended ID.
Class	Implementation Specific Parameter
Range	0 - 255
Default	0xF0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	Format C CONST(Can_RxFifoTableIdConfigType, CAN_CONST) RxFifoTableID_PBCfg [CAN_MAXTABLEID]= { {0x11121314, /* CanRxFifoTable_0 of formatC for FlexCAN_A */ 0xffffffff }}
Autosar 4.0 Requirement	NA

Note

Specifies an ID to be used as acceptance criteria for the ID Table 2.

3.10.4.5 CanIDValue3**Table 3-93. CanIDValue3**

Description	Specifies an ID to be used as acceptance criteria for the ID Table3. Value for this parameter should be entered based on the CanControllerIDAcceptanceMode selected as explained below C- One 8 most significant bit value of Standard or Extended ID.
Class	Implementation Specific Parameter

Table continues on the next page...

Table 3-93. CanIDValue3 (continued)

Range	0 - 255
Default	0xF0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	Format C CONST(Can_RxFifoTableIdConfigType, CAN_CONST) RxFifoTableID_PBCfg[CAN_MAXTABLEID]= { {0x11121314, /* CanRxFifoTable_0 of formatC for FlexCAN_A */ 0xffffffff } }
Autosar 4.0 Requirement	NA

Note

Specifies an ID to be used as acceptance criteria for the ID Table 3.

3.10.4.6 CanMBFilterMaskValue**Table 3-94. CanMBFilterMaskValue**

Description	Specifies filter mask value to be used as acceptance criteria for the ID Table. Note: Value for this parameter should be entered based on the CanControllerIDAcceptanceMode selected as explained below: A- Filtermask value for One full standard or Extended ID based on the CanTableIDType selected. B- Filtermask value for CanIDValue0(bit field : 0-10 for standard ID type and 0-13 for extended ID type) and CanIDValue1 (bit field : 11- 21 for standard ID type and 14-27 for extended ID type) C- Filtermask value for CanIDValue0(bit field : 0-7), CanIDValue1(bit field : 8-15), CanIDValue2(bit field : 16-23) and CanIDValue3(bit field : 24-31).
Class	Implementation Specific Parameter
Range	0 – 2047 for Format A Standard CanTableIDType 0 - 536870911 Format A Extended CanTableIDType 0 – 268435455 for Format B Extended CanTableIDType 0 - 4194303 for Format B Standard CanTableIDType 0 – 4294967295 for Format C
Default	0xF0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	CONST(Can_RxFifoTableIdConfigType, CAN_CONST) RxFifoTableID_PBCfg[CAN_MAXTABLEID]= { {0x11121314, /* CanRxFifoTable_0 of formatC for FlexCAN_A */, 0xFFFFFFFF } }
Autosar 4.0 Requirement	NA

3.10.4.7 CanTableIDType

Table 3-95. CanTableIDType

Description	Specifies whether extended or standard frames are accepted into the FIFO.
Class	Implementation Specific Parameter
Range	Standard, Extended
Default	Standard
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	NA
Autosar 4.0 Requirement	NA

3.10.5 CanControllerBaudrateConfig

CanControllerBaudRate

Name  CanControllerBaudRate_0

General

Can Time Segments Checking

 ☒ 

Can Bus Length (meters) (1 -> 5000)

 40

Can Propagation Delay Transceiver (ns) (0 -> 5000)

 150.0

Can Controller Prescaler (1 -> 256)

 100 

Can Controller Prescaler Alternate (1 -> 256)

 100

Can Controller BaudRate (Kbps) (0 -> 2000)

 20 

Can Propagation Segment (0 -> 7)

 5 

Can Phase Segment 1 (0 -> 7)

 5 

Can Phase Segment 2 (1 -> 7)

 6 

Can Resynch Jump Width (0 -> 3)

 0 

Can Automatic Time Segments Calculation

 ☐ 

Figure 3-12. CanControllerBaudrateConfig

Note

Can-Controller Baudrate Config parameters, their possible values and meaning are described in the following text.

The Can-Controller BaudRate parameters are implemented as constant structures and arrays stored in flash memory of the MCU.

3.10.5.1 CanControllerCheckCanStandard

Table 3-96. CanControllerCheckCanStandard

Description	If enabled, Can Plugin checks that CanControllerPropSeg, CanControllerSeg1, CanControllerSeg2 and CanSyncJumpWidth settings match the CAN Standard Compliant Bit Time Segment Settings.
Class	Implementation Specific Parameter.
Range	True, False
Default	True
Source File	NA
Source Representation	NA
Autosar 4.0 Requirement	NA

Note

This parameter is used in the Can_Cfg.c and Can_PBcfg.c files validating time segments values if it is enabled.

3.10.5.2 CanAdvancedSetting

Table 3-97. CanAdvancedSetting

Description	If True initiates the derivation of the Can bit timing values from the CanControllerBaudRate parameter and source clock value.
Class	Implementation Specific Parameter
Range	True, False
Default	False
Source File	Can_Cfg.c, Can_PBcfg.c
Source Representation	NA
Autosar 4.0 Requirement	NA

Note

If this parameter is set to “True” then “CanControllerPropSeg”, “CanControllerSeg1”, “CanControllerSeg2” and “CanSyncJumpWidth” are disabled and these values are calculated indirectly. In the same time two another parameters are enabled: "BusLength" and "PropagationDelayOfTranceiver".

3.10.5.3 CanBusLength

Table 3-98. CanBusLength

Description	Specifies the Can bus length in meters. This parameter is used for PropSeg calculation when “CanAdvancedSetting” is set to true.
Class	Implementation Specific Parameter
Range	1 – 5000
Default	40
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	NA
Autosar 4.0 Requirement	NA

Note

This parameter is available only if “AdvancedSetting” is set to true.

3.10.5.4 CanPropDelayTranceiver

Table 3-99. PropagationDelayOfTranceiver

Description	Specifies the propagation delay in ns for the Can transceiver. This parameter is used for PropSeg calculation when “CanAdvancedSetting” is set to true.
Class	Implementation Specific Parameter
Range	1 – 5000
Default	150
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	NA
Autosar 4.0 Requirement	NA

Note

This parameter is available only if “CanAdvancedSetting” is set to true.

3.10.5.5 CanControllerPrescaller

Table 3-100. CanControllerPrescaller

Description	Specifies the prescaler for the controller. The calculation of the resulting CanControllerTimeQuanta value depending on module clocking and prescaler shall be done offline. $\text{Prescaler} = \text{FreqCanClk} / \text{FreqTq}$; $\text{FreqTq} = 1 / \text{CanControllerTimeQuanta}$.
Class	Implementation Specific Parameter
Range	1 - 256

Table continues on the next page...

Table 3-100. CanControllerPrescaler (continued)

Default	100
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	NA
Autosar 4.0 Requirement	NA

3.10.5.6 CanControllerPrescaler_Alt

Table 3-101. CanControllerPrescaler_Alt

Description	Specifies the alternate prescaler for the controller .The calculation of the resulting CanControllerTimeQuanta_Alternate value depending on module clocking and prescaler shall be done offline.Prescaler = FreqCanClk / FreqTq; FreqTq = 1 / CanControllerTimeQuanta .
Class	Implementation Specific Parameter
Range	1 - 256
Default	100
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	NA
Autosar 4.0 Requirement	NA

Note

This parameter is available/editable only if "Can.CanConfig.DualClockMode" is set to STD_ON from Resource files and "CanEnableDualClockMode" is set to true.

3.10.5.7 CanControllerBaudRate

Table 3-102. CanControllerBaudRate

Description	Specifies the buadrate of the controller in kbps.
Class	Autosar Parameter
Range	0 - 1000
Default	20
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	NA
Autosar 4.0 Requirement	CAN005_Conf

3.10.5.8 CanControllerPropSeg

Table 3-103. CanControllerPropSeg

Description	Propogation delay in time quanta.
Class	Autosar Parameter
Range	0 - 7
Default	5
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[PROPSEG] - Propagation segment */ 5U , } </pre>
Autosar 4.0 Requirement	CAN073_Conf

Note

It is used to compensate the physical delay within the CAN network (CTRL[PROPSEG] - 0..7).

3.10.5.9 CanControllerSeg1

Table 3-104. CanControllerSeg1

Description	Specifies Phase Segment 1
Class	Autosar Parameter
Range	0 - 7
Default	5
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[PSEG1] - Segment 1 */ 5U , } </pre>
Autosar 4.0 Requirement	CAN074_Conf

Note

Specifies the Phase Segment 1 in time quantas ($CTRL[PSEG1] = 0..7$). $PHASE_BUF_SEG1 = (PSEG1 + 1) * Tq$. The $PHASE_BUF_SEG1$ valid values are 1-8 Tq .

3.10.5.10 CanControllerSeg2**Table 3-105. CanControllerSeg2**

Description	Specifies Phase Segment 2
Class	Autosar Parameter
Range	1 - 7
Default	6
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { /* ===== Control Register - CTRL ===== */ /* CTRL[PSEG2] - Segment 2 */ 6U , } </pre>
Autosar 4.0 Requirement	CAN075_Conf

Note

Specifies Phase Segment 2 in time quantum ($CTRL[PSEG2] = 1..7$). $PHASE_BUF_SEG2 = (PSEG2 + 1) * Tq$. The $PHASE_BUF_SEG2$ valid values are 2-8 Tq .

3.10.5.11 CanSyncJumpWidth**Table 3-106. CanSyncJumpWidth**

Description	Specifies Synchronization Jump Width.
Class	Implementation Specific Parameter
Range	0 - 3
Default	0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_ControllerConfigType, CAN_CONST) ControllerConfigs_PC[CAN_MAXCONTROLLERCOUNT] = { </pre>

Table continues on the next page...

**Table 3-106. CanSyncJumpWidth
(continued)**

	<pre> /* ===== Control Register - CTRL ===== */ /* CTRL[RJW] - Resynchronization Jump Width */ } </pre>
Autosar 4.0 Requirement	CAN383_Conf

Note

Specifies Synchronization Jump Width: CTRL[RJW] = 0..3.

3.10.6 Can HardwareObject**CanHardwareObject**

The screenshot shows the 'CanHardwareObject' configuration dialog. The 'Name' field contains 'CanA0_RX0'. The 'General' tab is selected, displaying the following settings:

- Can Implementation Type: BASIC
- Can ID Message Type: STANDARD
- Can ID Message Value: 0x1
- Can ID Bits Local Priority (0 -> 7): 0
- Can Object ID (MB Handle): 0
- Can MB Type: RECEIVE
- Can Controller Reference: /Can/Can/CanConfigSet_0/CanA0
- Can Filter Mask Reference: /Can/Can/CanConfigSet_0/CanA0/CanFilterMask_0

Figure 3-13. Can HardwareObject**Note**

When CanControllerRxFifoEnable for a controller is set to true then maximum number of hardware objects to be configured for that controller is 56.

When the FEN bit is set in the MCR register, the memory area from 0x80 to 0xFC (which is normally occupied by MBs 0 to 7) is used by the reception FIFO engine.

For reading data received from Fifo it should be used as the reading from MB0.

The MBs configuration at Can_Init() level will start to configure classic MBs from the MB index 8, because the space of MBs 0 to 7 is reserved for Rx Fifo.

3.10.6.1 CanHandleType

Table 3-107. CanHandleType

Description	Specifies the type (Full-Can or Basic-Can) of the hardware object.
Class	Autosar Parameter
Range	BASIC, FULL
Default	BASIC
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_MessageBufferConfigObjectType, CAN_CONST) MessageBufferConfigs0[] = { /* MessageBufferConfigs_PB[0], "CanA0_RX0" * /* IdMaskIndex */ 0U, /* ControllerId - based on the order from CanController list */ 0U, /* ID type: EXTENDED, STANDARD, MIXED */ STANDARD, /* Receive/Transmit MB configuration */ RECEIVE, /* MessageId */ 0x1U, /* Local priority bits used for arbitration */ 0U, </pre>
Autosar 4.0 Requirement	CAN323_Conf

3.10.6.2 CanIdType

Table 3-108. CanIdType

Description	Specifies whether the IdValue is of type: standard identifier, extended identifier, mixed mode.
Class	Autosar Parameter
Range	Standard, Mixed, Extended
Default	Standard

Table continues on the next page...

Table 3-108. CanIdType (continued)

Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_MessageBufferConfigObjectType, CAN_CONST) MessageBufferConfigs0[] = { /* MessageBufferConfigs_PB[0], "CanA0_RX0" * /* IdMaskIndex */ 0U, /* ControllerId - based on the order from CanController list */ 0U, /* ID type: EXTENDED, STANDARD, MIXED */ STANDARD, /* Receive/Transmit MB configuration */ RECEIVE, /* MessageId */ 0x1U, /* Local priority bits used for arbitration */ 0U, </pre>
Autosar 4.0 Requirement	CAN065_Conf

Note

Specifies whether the IdValue is of type: - standard identifier (ID - 11 bits length), - extended identifier (ID - 29 bits length), - mixed mode (standard or extended).

3.10.6.3 CanIdValue**Table 3-109. CanIdValue**

Description	Specifies (together with the filter mask) the identifiers that pass the hardware filter for of RX objects.
Class	Autosar Parameter
Range	0 .. 4294967295
Default	0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_MessageBufferConfigObjectType, CAN_CONST) MessageBufferConfigs0[] = { /* MessageBufferConfigs_PB[0], "CanA0_RX0" * /* IdMaskIndex */ 0U, /* ControllerId - based on the order from CanController list */ 0U, /* ID type: EXTENDED, STANDARD, MIXED */ </pre>

Table continues on the next page...

Table 3-109. CanIdValue (continued)

	STANDARD, /* Receive/Transmit MB configuration */ RECEIVE, /* MessageId */ 0x1U, /* Local priority bits used for arbitration */ 0U,
Autosar 4.0 Requirement	CAN325_Conf

Note

Specifies (together with the filter mask)- the identifiers range that passes the hardware filter for of RX objects. Parameter ranges from 0 to 0x7FF (11 bits) for Standard IDs and 0 to 0xFFFFFFFF (29 bits) for Extended IDs. User can assign any code to this parameter, but must to respect the above rule related to Standard/Extended IDs.

3.10.6.4 CanMBPrio**Table 3-110. CanMBPrio**

Description	This field is used when MCR[LPRIO_EN] is set and makes sense only for Tx MBs.
Class	Implementation Specific Parameter
Range	0 - 7
Default	0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	CONST(Can_MessageBufferConfigObjectType, CAN_CONST) MessageBufferConfigs0[] = { /* MessageBufferConfigs_PB[0], "CanA0_RX0" * /* IdMaskIndex */ 0U, /* ControllerId - based on the order from CanController list */ 0U, /* ID type: EXTENDED, STANDARD, MIXED */ STANDARD, /* Receive/Transmit MB configuration */ RECEIVE, /* MessageId */ 0x1U, /* Local priority bits used for arbitration */

Table continues on the next page...

Table 3-110. CanMBPrio (continued)

	0U,
Autosar 4.0 Requirement	NA

Note

MBCS[PRIO]: Local priority. This 3-bit field is used when MCR[LPRIO_EN] is set and makes sense only for TX buffers. If CTRL[LBUF] is set this field is not used. These bits are not transmitted. They are appended to the regular ID to define the transmission priority.

3.10.6.5 CanObjectId**Table 3-111. CanObjectId**

Description	Holds the handle ID of Hrh or Hth. The value of this parameter is unique in a given CAN Driver and should start with 0 and continue without any gaps. The Hrh and Hth IDs are defined under two different name-spaces. Examples: Hrh0-0, Hrh1-1, Hth0-2, Hth1-3
Class	Autosar Parameter
Range	0 .. 65535
Default	0
Source File	Can_Cfg.h
Source Representation	#define CanA0_RX0 0U /* RECEIVE object */
Autosar 4.0 Requirement	CAN326_Conf

Note

Holds the handle ID of HRH or HTH.

3.10.6.6 CanObjectType**Table 3-112. CanObjectType**

Description	Specifies if the HardwareObject is used as Transmit or as Receive object
Class	Autosar Parameter
Range	Transmit, Receive
Default	Receive
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	CONST(Can_MessageBufferConfigObjectType, CAN_CONST) MessageBufferConfigs0[] = { /* MessageBufferConfigs_PB[0], "CanA0_RX0" * /* IdMaskIndex */ 0U,

Table continues on the next page...

Table 3-112. CanObjectType (continued)

	/* ControllerId - based on the order from CanController list */ 0U, /* ID type: EXTENDED, STANDARD, MIXED */ STANDARD, /* Receive/Transmit MB configuration */ RECEIVE, /* MessageId */ 0x1U, /* Local priority bits used for arbitration */ 0U,
Autosar 4.0 Requirement	CAN327_Conf

Note

Specifies if the HardwareObject is used as Transmit or as Receive object.

3.10.6.7 CanControllerRef**Table 3-113. CanControllerRef**

Description	This associates the hardware object to the CAN controller that uses this hardware object.
Class	Autosar Parameter
Range	Integer
Default	0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	CONST(Can_MessageBufferConfigObjectType, CAN_CONST) MessageBufferConfigs0[] = { /* MessageBufferConfigs_PB[0], "CanA0_RX0" * /* IdMaskIndex */ 0U, /* ControllerId - based on the order from CanController list */ 0U, /* ID type: EXTENDED, STANDARD, MIXED */ STANDARD, /* Receive/Transmit MB configuration */ RECEIVE, /* MessageId */ 0x1U, /* Local priority bits used for arbitration */ 0U,

Table continues on the next page...

Table 3-113. CanControllerRef (continued)

Autosar 4.0 Requirement	CAN322_Conf
--------------------------------	-------------

Note

Reference to CAN Controller to which the HOH is associated to.

3.10.6.8 CanFilterMaskRef**Table 3-114. CanFilterMaskRef**

Description	Describes a mask for hardware-based filtering of CAN identifiers. It shall be distinguished between: - Standard identifier mask, - Extended identifier mask
Class	Autosar Parameter
Range	Integer
Default	0
Source File	Can_Cfg.c, Can_PBCfg.c
Source Representation	<pre> CONST(Can_MessageBufferConfigObjectType, CAN_CONST) MessageBufferConfigs0[] = { /* MessageBufferConfigs_PB[0], "CanA0_RX0" * /* IdMaskIndex */ 0U, /* ControllerId - based on the order from CanController list */ 0U, /* ID type: EXTENDED, STANDARD, MIXED */ STANDARD, /* Receive/Transmit MB configuration */ RECEIVE, /* MessageId */ 0x1U, /* Local priority bits used for arbitration */ 0U, </pre>
Autosar 4.0 Requirement	CAN321_Conf

Note

Reference to the filter mask that is used for hardware filtering together with the CAN_ID_VALUE. This value is used as acceptance masks for ID filtering in RX MBs and the FIFO.

3.10.6.9 CanMainFunctionRWPeriodRef

Table 3-115. CanMainFunctionRWPeriodRef

Description	Reference to CAN Controller to which the HOH is associated to.
Class	Autosar Parameter
Range	NA
Default	NA
Container Name	CanMainFunctionRWPeriods{CAN_MAIN_FUNCTION_RWPERIODS}
Autosar 4.0 Requirement	CAN437_Conf

Note

This parameter describes the period for cyclic call to Can_MainFunction_Write and Can_MainFunction_Read . Unit is seconds. Different poll-cycles will be configurable if more than one CanMainFunctionWritePeriod or Can_MainFunction_ReadPeriod are configured. In this case multiple Can_MainFunction_Write() or Can_MainFunction_Read() will be provided by the CAN Driver module.

The Can driver configuration shall deviate from AUTOSAR defined configuration of "Main function RW periods" to make them usable:

- 1) Container CanMainFunctionRWPeriods (ECUC_Can_00437) contains 0..1 CanMainFunctionReadPeriod (ECUC_Can_00356) and 0..1 CanMainFunctionWritePeriod (ECUC_Can_00358), but only one of both within each container instance
- 2) Container CanGeneral (ECUC_Can_00328) contains 1..* CanMainFunctionRWPeriods (its "included containers")
- 3) CanMainFunctionRWPeriodRef (ECUC_Can_00438) refers (as specified by CAN438_Conf) container CanMainFunctionRWPeriods, but not parameter CanMainFunctionReadPeriod and not parameter CanMainFunctionWritePeriod.

3.10.7 Can MainFunction Read Period Parameters

CanMainFunctionReadPeriod parameters, their possible values and meaning are described in the following text. CanMainFunctionReadPeriod parameters are implemented as preprocessor defines.

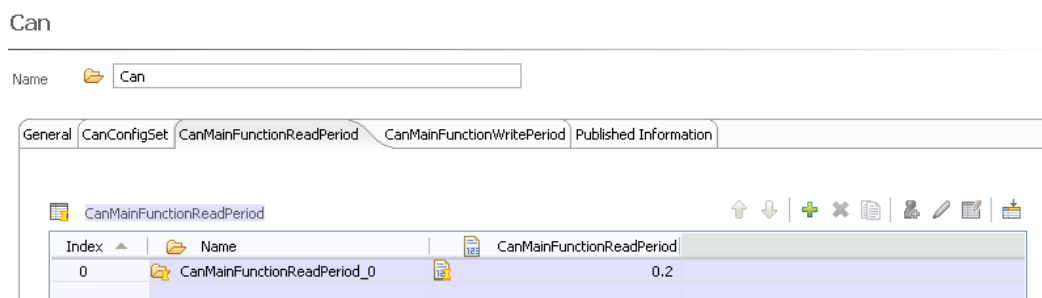


Figure 3-14. Can MainFunction Read Period Parameters

3.10.7.1 CanMainFunctionReadPeriod

Table 3-116. CanMainFunctionReadPeriod

Description	This parameter describes the period for cyclic call to Can_MainFunction_Read. Unit is seconds.
Class	Autosar Parameter
Range	0.001 .. 65.535
Default	NA
Source File	Can_Cfg.h
Source Representation	#define CAN_MAINFUNCTION_PERIOD_READ 0.001U
Autosar 4.0 Requirement	CAN356_Conf

Note

Different poll-cycles will be configurable if more than one CanMainFunctionReadPeriod is configured. In this case multiple Can_MainFunction_Read() will be provided by the CAN Driver module.

3.10.8 Can MainFunction Write Period Parameters

CanMainFunctionWritePeriod parameters, their possible values and meaning are described in the following text. CanMainFunctionWritePeriod parameters are implemented as preprocessor defines.

Configuration Parameters

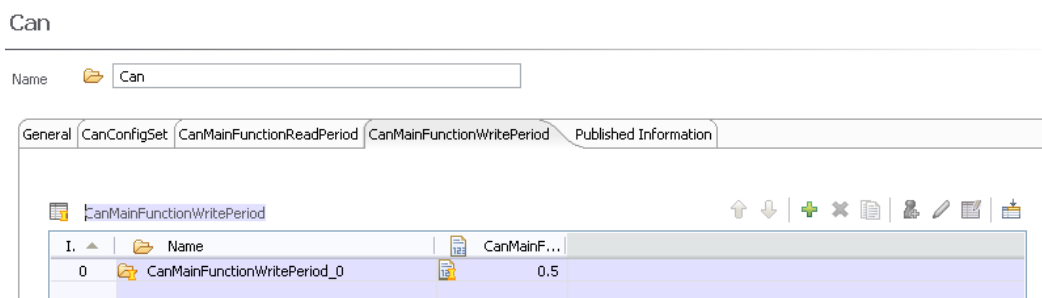


Figure 3-15. Can MainFunction Write Period Parameters

3.10.8.1 CanMainFunctionWritePeriod

Table 3-117. CanMainFunctionWritePeriod

Description	This parameter describes the period for cyclic call to Can_MainFunction_Write. Unit is seconds.
Class	Autosar Parameter
Range	0.001 .. 65.535
Default	NA
Source File	Can_Cfg.h
Source Representation	#define CAN_MAINFUNCTION_PERIOD_WRITE 0.001U
Autosar 4.0 Requirement	CAN358_Conf

Note

Different poll-cycles will be configurable if more than one CanMainFunctionWritePeriod is configured. In this case multiple Can_MainFunction_Write() will be provided by the CAN Driver module.

3.10.9 Can Common Published Information

Can

Name  Can

General CanConfigSet Published Information

▼ **CommonPublishedInformation**

 CommonPublishedInformation

AUTOSAR Major Version  2

AUTOSAR Minor Version  2

AUTOSAR Patch Version  2

Numeric Module ID  80

Software Major Version  0

Software Minor Version  9

Software Patch Version  2

Vendor Api Infix  0

Vendor ID  43

Figure 3-16. Can Common Published Information

3.10.9.1 ArReleaseMajorVersion

Table 3-118. ArReleaseMajorVersion

Description	Major version number of AUTOSAR specification on which the appropriate implementation is based on.
Class	Implementation Specific Parameter
Range	Integer
Default	M4_SRC_AR_SPEC_VERSION_MAJOR
Source File	All
Source Representation	ARVersion M4_SRC_AR_SPEC_VERSION_MAJOR .M4_SRC_AR_SPEC_VERSION_MINOR.M4_SRC_AR_SPEC_VERSION_PATCH
Autosar 4.0 Requirement	NA

3.10.9.2 ArReleaseMinorVersion

Table 3-119. ArReleaseMinorVersion

Description	Minor version number of AUTOSAR specification on which the appropriate implementation is based on.
Class	Implementation Specific Parameter
Range	Integer
Default	M4_SRC_AR_SPEC_VERSION_MINOR
Source File	All
Source Representation	ARVersion M4_SRC_AR_SPEC_VERSION_MAJOR.M4_SRC_AR_SPEC_VERSION_MINOR.M4_SRC_AR_SPEC_VERSION_PATCH
Autosar 4.0 Requirement	NA

3.10.9.3 ArReleaseRevisionVersion

Table 3-120. ArReleaseRevisionVersion

Description	Revision level version number of AUTOSAR specification on which the appropriate implementation is based on.
Class	Implementation Specific Parameter
Range	Integer
Default	M4_SRC_AR_SPEC_VERSION_PATCH
Source File	All
Source Representation	ARVersion M4_SRC_AR_SPEC_VERSION_MAJOR.M4_SRC_AR_SPEC_VERSION_MINOR.M4_SRC_AR_SPEC_VERSION_PATCH
Autosar 4.0 Requirement	NA

3.10.9.4 ModuleId

Table 3-121. ModuleId

Description	Module ID of this module from Module List.
Class	Implementation Specific Parameter
Range	Integer
Default	M4_XDM_AR_MODULE_ID
Source File	NA
Source Representation	NA
Autosar 4.0 Requirement	NA

3.10.9.5 SwMajorVersion

Table 3-122. SwMajorVersion

Description	Major version number of the vendor specific implementation of the module. The numbering is vendor specific.
Class	Implementation Specific Parameter
Range	Integer
Default	M4_SRC_SW_VERSION_MAJOR
Source File	All
Source Representation	@version M4_SRC_SW_VERSION_MAJOR.M4_SRC_SW_VERSION_MINOR.M4_SRC_SW_VERSION_PATCH
Autosar 4.0 Requirement	NA

3.10.9.6 SwMinorVersion

Table 3-123. SwMinorVersion

Description	Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.
Class	Implementation Specific Parameter
Range	Integer
Default	M4_SRC_SW_VERSION_MINOR
Source File	All
Source Representation	@version M4_SRC_SW_VERSION_MAJOR.M4_SRC_SW_VERSION_MINOR.M4_SRC_SW_VERSION_PATCH
Autosar 4.0 Requirement	NA

3.10.9.7 SwPatchVersion

Table 3-124. SwPatchVersion

Description	Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.
Class	Implementation Specific Parameter
Range	Integer
Default	M4_SRC_SW_VERSION_PATCH
Source File	All

Table continues on the next page...

Table 3-124. SwPatchVersion (continued)

Source Representation	@version M4_SRC_SW_VERSION_MAJOR.M4_SRC_SW_VERSION_MINOR.M4_SRC_SW_VERSION_PATCH
Autosar 4.0 Requirement	NA

3.10.9.8 VendorApilnfix

Table 3-125. VendorApilnfix

Description	Vendor API.
Class	Implementation Specific Parameter
Range	Integer
Default	NA
Source File	NA
Source Representation	NA
Autosar 4.0 Requirement	NA

3.10.9.9 VendorId

Table 3-126. VendorId

Description	Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.
Class	Implementation Specific Parameter
Range	Integer
Default	M4_SRC_AR_MODULE_VENDOR_ID
Source File	All
Source Representation	#define CAN_VENDOR_ID_C M4_SRC_AR_MODULE_VENDOR_ID
Autosar 4.0 Requirement	NA

3.11 Wakeup Configuration

The current driver implementation is using one form of wakeup event detection and handling: Internal Wakeup.

Internal Wakeup is enabled by the option set in the Resource files:

Can.CanConfig.InternalWakeupSupport:STD_ON

Valid values for this parameter are STD_ON / STD_OFF.

Note

MPC5604B and MPC5607B doesn't support internal wakeup functionality. INTC table has the CAN wakeup interrupt reserved.

Wakeup Implementation

General Wake-Up mechanism can be enabled or disabled by the “CanWakeupSupport” control under "CanController" container. This is the implementation of the CAN330_Conf requirement. See below the definition of the macro that enables/ disables this functionality (located in Can_Cfg.h file).

```
#define CAN_WAKEUP_SUPPORT (STD_ON)
```

The CAN Controller can handle wakeup events by interrupts or by polling mode. The selection is done by the precompile parameter “CanWakeupProcessing” that is specific to every controller.

a. Interrupt Mode

ISR(Can_IsrFCx_WKP) is the function that handles with the wakeup events. Global Wakeup parameter must be enabled and also the specific controller must have enabled the support of interrupts.

```
#if (CAN_WAKEUP_SUPPORT == STD_ON)
    #if (CAN_A_WAKEUPINT_SUPPORTED == STD_ON)
        #endif
    #endif
```

b. Polling Mode

Can_MainFunction_Wakeup is the function that handles with the wakeup events in polling mode. Global Wakeup parameter must be enabled and also the specific controller must have enabled the support of wakeup polling mode.

```
#if (CAN_WAKEUP_SUPPORT == STD_ON)
    #if (CAN_WAKEUPPOLL_SUPPORTED == STD_ON)
        #endif
    #endif
```

3.12 Configuration Exporting

If PreCompile support is enabled, then Can_Cfg.h file will contains the declaration of configuration structure:

```
#ifdef CAN_PRECOMPILE_SUPPORT
#define CAN_INIT_CONFIG_PC_DEFINES \
    extern CONST(Can_ConfigType, CAN_CONST) Can_ConfigSet_PC;
#endif
```

If PostBuild support is enabled, then Can_Cfg.h file will contains the declaration of configuration structures:

```
#ifndef CAN_PRECOMPILE_SUPPORT
#define CAN_INIT_CONFIG_PB_DEFINES \
    extern CONST(Can_ConfigType, CAN_CONST) CanConfigSet_0_PB; \
    extern CONST(Can_ConfigType, CAN_CONST) CanConfigSet_1_PB;
#endif
```

Can.h file will export this configuration structures:

```
#ifdef CAN_PRECOMPILE_SUPPORT
    CAN_INIT_CONFIG_PC_DEFINES
#else /* CAN_PRECOMPILE_SUPPORT */
    CAN_INIT_CONFIG_PB_DEFINES
#endif /* CAN_PRECOMPILE_SUPPORT */
```

3.13 Rx Fifo

The receive-only FIFO is enabled by asserting the FEN bit in the MCR register. The Rx Fifo configuration in the TRESOS plugin is implemented by “CanControllerRx Fifo Enable” parameter under “CanController” container.

When the Fifo is enabled, the memory region normally occupied by the first 8 MBs (\$80 - \$FF) is normally reserved for use of the Fifo engine. The CPU can read the received frames sequentially, in the order they were received, by repeatedly accessing the MB0 structure.

The interrupts corresponding to MB0 to 7 have a different behavior when Rx Fifo is enabled. Bit 7 of the IFLAG1 becomes the “Fifo Overflow” flag, bit 6 becomes the “Fifo Warning” flag, bit 5 becomes the “Frame Available in Fifo” flag and bits 4 to 0 are unused.

Note: The hardware objects configured in the “Can_MBConfigObjectType” structure are used for the initialization of MBs. If Rx Fifo is enabled then the MB initialization will start from the MB8 because first 8 MBs (0 to 7) are reserved for the Rx Fifo. Also the total number of hardware objects per controller is reduced to 56 from 64 when Rx Fifo is enabled.

If Rx Fifo is enabled the user can define proper handlers for overflow and warnings notification events. If the precompile parameter “CanCodeSizeOptimization” is set true or Rx processing is configured in polling mode, the 3 events of Fifo (overflow, warning and frame available) are processed by the same low level driver routine (see Can_LLD_Process_Rx).

If user configures FIFO support for a CAN controller then it is the responsibility of the USER to configure minimum one HW object of type “Receive” belonging to that controller, so that the first configured HW object of type receive will be used as a reference for the FIFO of that particular controller

3.14 Driver Usage and Configuration Tips

1. A CAN Hardware Unit consists of one or multiple CAN controllers of the same type. Can_MainFunction_Write(), Can_MainFunction_Read(), Can_MainFunction_BusOff() and Can_MainFunction_Wakeup() APIs are defined if at least one Can controller from the Hardware Unit is configured to Polling mode for write, read, busoff and wakeup operation - else are empty functions. Refer to CAN178, CAN180, CAN183 and CAN185.
2. Can_CheckWakeup() and Can_MainFunction_Wakeup() APIs are not defined if Wakeup support is disabled.
3. Can_AbortMb() API (Non Autosar) is defined if this feature is enabled from the Tresos plugin.
4. Can_SetClockMode() API (Non Autosar) is defined if this feature is enabled from the Tresos plugin.
5. The CAN Hardware Unit can be initialized using Can_Init() API.
6. A single CAN controller can be initialized using Can_InitController() API. The condition is that controller should be in STOP state (not participating to bus communication) while it is initialized.
7. Every CAN controller initialization is preceded by a software reset. See Can_LLD_ResetController() routine from low level driver.

- 8.** Can_InitController() API should configure the MCR, CTRL, RXIMR registers, RxFifo block structure (if enabled), Message Buffers (for Rx MB is configured also every RXIMR register - used for message filtering).
- 9.** Can_LLD_SetControllerMode() API can use transitions to CAN_T_SLEEP and CAN_T_WAKEUP only if Wakeup feature is enabled or supported by the platform.
- 10.** Interrupts can be enabled calling Can_EnableControllerInterrupts() only if Can_DisableControllerInterrupts() was called prior. Refer to CAN208.
- 11.** Multiplex transmission (supported by Can_Write() API) means to send a message from any Tx MB that is free to be used, in the range of the same HWObjectID. This means that several Hardware Objects can have the same HWObjectID. This feature can be used only if it's enabled from the Tresos Plugin. Refer to CAN277.
- 12.** For Tx MBs the difference between Standard and Extended mode is done by the most significant bit of the Can ID. Refer to CANIF243 and CANIF188.
- 13.** For Rx MBs the MIXED message buffer type is handled as EXTENDED type. Based on the MB type the RXIMR register is configured according: for STANDARD type the value is left shift with 18 bits.

Chapter 4

The Configuration of Can Bit Timing

4.1 Clock Source Description

The FlexCAN module supports a variety of means to setup bit timing parameters that are required by the CAN protocol. The Control Register has various fields used to control bit timing parameters: PRESDIV, PROPSEG, PSEG1, PSEG2 and RJW.

The PRESDIV field (CTRL[PRESDIV]) controls a prescaler that generates the Serial Clock (Sclock), whose period defines the ‘time quantum’ used to compose the CAN waveform. A time quantum is the atomic unit of time handled by the CAN engine.

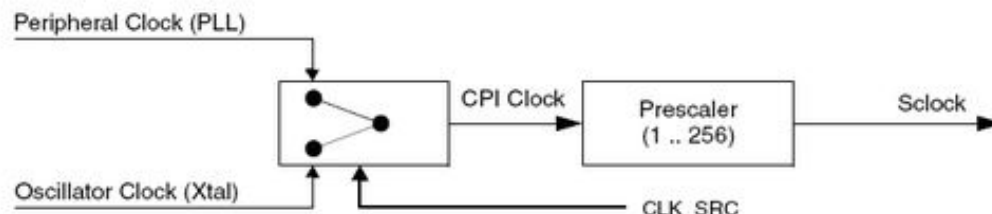


Figure 4-1. Can Engine Clock Scheme

A bit time is subdivided into three segments:

SYNC_SEG: This segment has a fixed length of one time quantum. Signal edges are expected to happen within this section.

Time Segment 1: This segment includes the Propagation Segment and the Phase Segment 1 of the CAN standard. It can be programmed by setting the PROPSEG and the PSEG1 fields of the CTRL Register so that their sum (plus 2) is in the range of 4 to 16 time quanta.

Time Segment 2: This segment represents the Phase Segment 2 of the CAN standard. It can be programmed by setting the PSEG2 field of the CTRL Register (plus 1) to be 2 to 8 time quanta long.

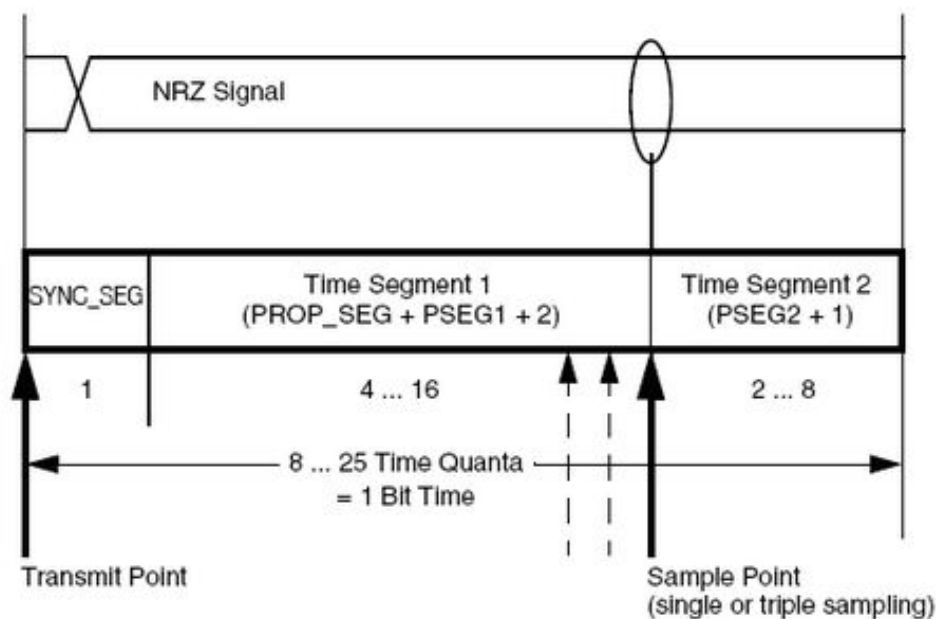


Figure 4-2. Can Time Segments

Understanding the Clock Selection for Can

The clock source for CAN module can be obtained in 2 modes, depending by the setting of the "CanClockFromBus" control in Tressos.



Figure 4-3. Can Clock Selection: bus

When "CanClockFromBus" is set to false the clock value is extracted from the "CanCrystalFrequencyHz" control of Can Tresos plugin.



Figure 4-4. Can Clock Selection: osc_clock

4.2 Time Segments Calculation

Basic-Can vs. Full-Can

The terms **Basic CAN** and **Full CAN** must not be confused with the terms Standard CAN - also known as Base Frame Format (11 bit identifier, Version 2.0A data format) and Extended CAN - also known as Extended Frame Format (29 bit identifier, or Version 2.0B data format). Suitably configured, each implementation (Basic or Full CAN) can handle both Base and Extended data formats.

NOTE: Can Driver doesn't make any difference between Full and Basic Hardware Object Type. This is implemented in the plugin configuration just because it's a requirement of Autosar (CAN323_Conf)

Time Segments Calculation

Every CanController container of Tressos configuration tool has a parameter named "CanClockFromBus" which selects the source clock from MCU configuration (if "CanClockFromBus" is true) or from "CanCrystalFrequencyHz" parameter of "CanGeneral" container (if "CanClockFromBus" is false). In both cases the value is expressed in Hertz.

Step.1

First step is to input the Prescaler (CTRL[PRES DIV]). Under the "Can Controller BaudRate" container, the Can Controller Prescaler must be completed. From this unit can calculate the "CanControllerTimeQuanta" and the frequency of the CanControllerQuanta (in hertz). These informations are needed for Automatic Time Segments Calculation.

CanControllerQuanta = 1 / CanControllerTimeQuanta (hertz = 1 / seconds)

Prescaler = CanClockFreq / CanControllerQuanta (constant = hertz / hertz)

NOTE: Valid interval for Prescaler is between 1 and 256.

Step.2

Second step is to calculate the number of CanTimeQuantas per Can bit. Autosar permit to to configure the BaudRate (in Kbps) using parameter "CanControllerBaudRate" from "CanControllerBaudRate" container of Tressos Can Plugin.

CanTimeQuanta = (1 / CancontrollerBaudRate) / CanControllerTimeQuanta (const = (1 / hertz) / seconds)

NOTE: Valid interval for No of CanTimeQuantas is between 8 and 25.

Step.3

Third step is to check the compatibility for parameters. The sum of Propagation Segment, Pseg1, Pseg2 and SyncSeg must be equal with no. of CanTimeQuantas resulted as above.

$$\text{No of Time Quantas per bit} = 1 + \text{PropSeg} + 1 + \text{Pseg1} + 1 + \text{Pseg2} + 1$$

4.3 CAN Bit Timing

Selecting bit timing parameters that work well on the bench may not equate with the situation that the product moves into the real environment (maximum wiring length, worst case configuration, oscillator tolerance, etc) and we can find that bit timing parameters are inadequate.

For the CAN protocol (J1939, J2284) is recommended to use a sample point in the range of 80% to 90%.

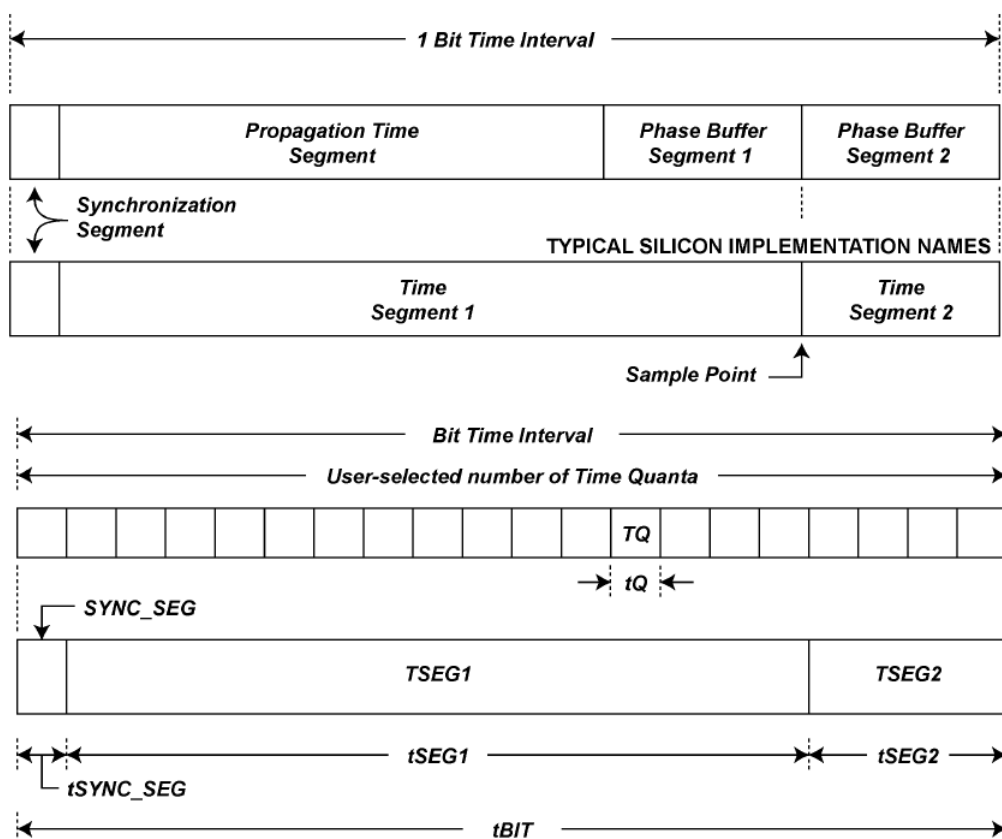


Figure 4-5. Can Bit Timing

The most important part of CAN bit timing is the Time Quantum (TQ). The time duration of the time quantum is derived from the CAN controller clock oscillator and the adjustable clock divider (prescaler). For Autosar CAN requirements we have to define Clock source and Time Quantum and calculate indirectly the prescaler.

The Synchronization Segment (SYNC_SEG) time interval is used to synchronize all the nodes across the network. SYNC_SEG time interval has a fixed period of one Time Quantum.

Time Segment 1 (TSEG1) is the time interval used to compensate for both positive phase errors in synchronization between nodes on the network and propagation delay between network nodes.

Time Segment 2 (TSEG2) is the time interval used to compensate for negative phase errors in synchronization between nodes.

Re-synchronization Jump Width (SJW) is not directly a segment of the bit time, but is used to dynamically adjust TSEG1 and TSEG2. SJW is the maximum amount of time by which TSEG1 may be lengthened or TSEG2 shortened to compensate for synchronization differences between nodes on the CAN network.

Bit Timing Example

```

Clock Source= 20MHz
Prescaler = 2
Bit Rate = 500K bits per second
From these input data will result:
    Time Bit = 2 microseconds ( 1/ 500Kbps)
    CAN clock source = 10 MHz (Clock Source / Prescaler)
    Selecting TQ = 100 nsec -> No. TQs per bit = 20 ( 2 usec / 100 nsec)
Bit Time Segments Calculation:
    SYNC_SEG = 1 (according to CAN ISO standard)
    TSEG1 = 15
    TSEG2 = 4
Sample Point Calculation:
    SP = (1 + TSEG1) / (1 + TSEG1 + TSEG2) = 0.8 -> 80%

```


Chapter 5

Interrupts Implementation

5.1 General Aspects

Autosar specifications permit to configure Can controller in interrupt mode for the following events: Rx Processing, Tx Processing, BusOff Processing, Wakeup Processing.

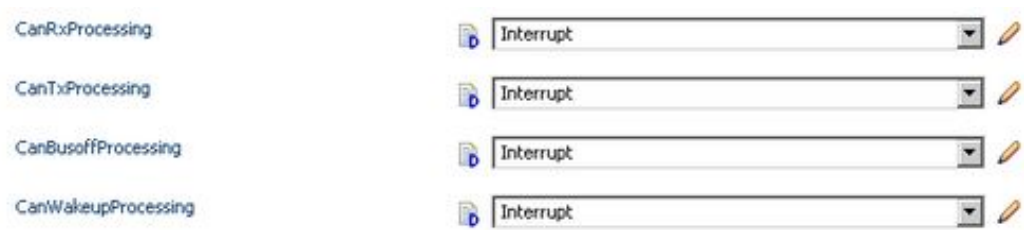


Figure 5-1. Can Interrupts Selection

Interrupts implementation and mapping is done according to MPC560XB Reference Manual.

Note

The interrupt service routines are not coded as re-entrant and may only be preempted by code which does not call any of the driver functions.

5.2 Interrupt Handlers

Table 5-1. Interrupt Handlers

Interrupt Vector Number	Interrupt Handler	Details
65	Can_IsrFCA_ERR	
66	Can_IsrFCA_BO	

Table continues on the next page...

Table 5-1. Interrupt Handlers (continued)

67	Reserved	
68	Can_IsrFCA_MB_00_03	1. Replaced by "Can_IsrFCA_UNI" if code size optimization is selected at pre-compile time. 2. Not used if RxFifo enabled.
69	Can_IsrFCA_MB_04_07	1. Replaced by "Can_IsrFCA_UNI" if code size optimization is selected at pre-compile time. 2. Replaced by "Can_IsrFCA_RxFifoEvents" if RxFifo is enabled.
70	Can_IsrFCA_MB_08_11	1. Replaced by "Can_IsrFCA_UNI" if code size optimization is selected at pre-compile time.
71	Can_IsrFCA_MB_12_15	1. Replaced by "Can_IsrFCA_UNI" if code size optimization is selected at pre-compile time.
72	Can_IsrFCA_MB_16_31	1. Replaced by "Can_IsrFCA_UNI" if code size optimization is selected at pre-compile time.
73	Can_IsrFCA_MB_32_63	1. Replaced by "Can_IsrFCA_UNI" if code size optimization is selected at pre-compile time.
85	Can_IsrFCB_ERR	
86	Can_IsrFCB_BO	
87	Reserved	
88	Can_IsrFCB_MB_00_03	1. Replaced by "Can_IsrFCB_UNI" if code size optimization is selected at pre-compile time. 2. Not used if RxFifo enabled.
89	Can_IsrFCB_MB_04_07	1. Replaced by "Can_IsrFCB_UNI" if code size optimization is selected at pre-compile time. 2. Replaced by "Can_IsrFCB_RxFifoEvents" if RxFifo is enabled.
90	Can_IsrFCB_MB_08_11	1. Replaced by "Can_IsrFCB_UNI" if code size optimization is selected at pre-compile time.
91	Can_IsrFCB_MB_12_15	1. Replaced by "Can_IsrFCB_UNI" if code size optimization is selected at pre-compile time.
92	Can_IsrFCB_MB_16_31	1. Replaced by "Can_IsrFCB_UNI" if code size optimization is selected at pre-compile time.
93	Can_IsrFCB_MB_32_63	1. Replaced by "Can_IsrFCB_UNI" if code size optimization is selected at pre-compile time.
105	Can_IsrFCC_ERR	
106	Can_IsrFCC_BO	
107	Reserved	
108	Can_IsrFCC_MB_00_03	1. Replaced by "Can_IsrFCC_UNI" if code size optimization is selected at pre-compile time. 2. Not used if RxFifo enabled.
109	Can_IsrFCC_MB_04_07	1. Replaced by "Can_IsrFCC_UNI" if code size optimization is selected at pre-compile time. 2. Replaced by "Can_IsrFCC_RxFifoEvents" if RxFifo is enabled.

Table continues on the next page...

Table 5-1. Interrupt Handlers (continued)

110	Can_IsrFCC_MB_08_11	1. Replaced by "Can_IsrFCC_UNI" if code size optimization is selected at pre-compile time.
111	Can_IsrFCC_MB_12_15	1. Replaced by "Can_IsrFCC_UNI" if code size optimization is selected at pre-compile time.
112	Can_IsrFCC_MB_16_31	1. Replaced by "Can_IsrFCC_UNI" if code size optimization is selected at pre-compile time.
113	Can_IsrFCC_MB_32_63	1. Replaced by "Can_IsrFCC_UNI" if code size optimization is selected at pre-compile time.
173	Can_IsrFCD_ESR	
174	Can_IsrFCD_BO	
175	Reserved	
176	Can_IsrFCD_MB_00_03	1. Replaced by "Can_IsrFCD_UNI" if code size optimization is selected at pre-compile time. 2. Not used if RxFifo enabled.
177	Can_IsrFCD_MB_04_07	1. Replaced by "Can_IsrFCD_UNI" if code size optimization is selected at pre-compile time. 2. Replaced by "Can_IsrFCD_RxFifoEvents" if RxFifo is enabled.
178	Can_IsrFCD_MB_08_11	1. Replaced by "Can_IsrFCD_UNI" if code size optimization is selected at pre-compile time.
179	Can_IsrFCD_MB_12_15	1. Replaced by "Can_IsrFCD_UNI" if code size optimization is selected at pre-compile time.
180	Can_IsrFCD_MB_16_31	1. Replaced by "Can_IsrFCD_UNI" if code size optimization is selected at pre-compile time.
181	Can_IsrFCD_MB_32_63	1. Replaced by "Can_IsrFCD_UNI" if code size optimization is selected at pre-compile time.
190	Can_IsrFCE_ERR	
191	Can_IsrFCE_BO	
192	Reserved	
193	Can_IsrFCE_MB_00_03	1. Replaced by "Can_IsrFCE_UNI" if code size optimization is selected at pre-compile time. 2. Not used if RxFifo enabled.
194	Can_IsrFCE_MB_04_07	1. Replaced by "Can_IsrFCE_UNI" if code size optimization is selected at pre-compile time. 2. Replaced by "Can_IsrFCE_RxFifoEvents" if RxFifo is enabled.
195	Can_IsrFCE_MB_08_11	1. Replaced by "Can_IsrFCE_UNI" if code size optimization is selected at pre-compile time.
196	Can_IsrFCE_MB_12_15	1. Replaced by "Can_IsrFCE_UNI" if code size optimization is selected at pre-compile time.
197	Can_IsrFCE_MB_16_31	1. Replaced by "Can_IsrFCE_UNI" if code size optimization is selected at pre-compile time.
198	Can_IsrFCE_MB_32_63	1. Replaced by "Can_IsrFCE_UNI" if code size optimization is selected at pre-compile time.
202	Can_IsrFCF_ERR	

Table continues on the next page...

Table 5-1. Interrupt Handlers (continued)

203	Can_IsrFCF_BO	
204	Reserved	
205	Can_IsrFCF_MB_00_03	1. Replaced by "Can_IsrFCF_UNI" if code size optimization is selected at pre-compile time. 2. Not used if RxFifo enabled.
206	Can_IsrFCF_MB_04_07	1. Replaced by "Can_IsrFCF_UNI" if code size optimization is selected at pre-compile time. 2. Replaced by "Can_IsrFCF_RxFifoEvents" if RxFifo is enabled.
207	Can_IsrFCF_MB_08_11	1. Replaced by "Can_IsrFCF_UNI" if code size optimization is selected at pre-compile time.
208	Can_IsrFCF_MB_12_15	1. Replaced by "Can_IsrFCF_UNI" if code size optimization is selected at pre-compile time.
209	Can_IsrFCF_MB_16_31	1. Replaced by "Can_IsrFCF_UNI" if code size optimization is selected at pre-compile time.
210	Can_IsrFCF_MB_32_63	1. Replaced by "Can_IsrFCB_UNI" if code size optimization is selected at pre-compile time.

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number UM13CANASR4.0R1.0.1
Revision 1.2